
ANATRA 1.1.0 User Guide

Kento Kasahara@The University of Osaka

2026 年 06 月 11 日

Contents:

第 1 章	ANATRA とは？	1
第 2 章	ANATRA のインストール	5
2.1	ANATRA を利用する上で必要となるプログラム	5
2.2	ソースコードからインストールする	5
2.2.1	HDF5 ライブラリを併せてビルドする場合	7
2.2.2	OpenBLAS を利用する場合	7
第 3 章	anatra の使い方	9
3.1	はじめに	9
3.2	共通オプション	12
3.3	トラジェクトリ変換 (trjconv)	15
3.3.1	オプション一覧	16
3.3.2	解析例	18
3.3.2.1	トラジェクトリから水を除去	19
3.3.2.2	PBC ラップ	19
3.3.2.3	フィッティング	19
3.3.2.4	センタリング	20
3.3.2.5	PBC ラップ&フィッティング	20
3.3.2.6	PBC ラップ&センタリング	21
3.4	距離 (distance, dis)	22
3.4.1	オプション一覧	22
3.4.2	出力ファイルのフォーマット	24
3.4.3	解析例	24
3.4.3.1	通常の原子間距離の計算	24
3.5	重心解析・平均二乗変位 (center_of_mass, com)	25
3.5.1	理論背景	25
3.5.1.1	平均二乗変位	25
3.5.2	オプション一覧	26
3.5.3	出力ファイルのフォーマット	27
3.5.4	解析例	28
3.5.4.1	タンパク質の MSD 計算	28
3.5.4.2	複数個存在する小分子の MSD 計算	28
3.6	動径分布関数 (radial_distr, rdf)	29
3.6.1	理論背景	29
3.6.1.1	同種粒子間の動径分布関数	29
3.6.1.2	異種粒子間の動径分布関数	30
3.6.2	オプション一覧	31

3.6.3	出力ファイルのフォーマット	32
3.6.4	解析例	32
3.6.4.1	水分子の酸素原子間の RDF	32
3.6.4.2	水分子の酸素原子-水素原子間の RDF	33
3.6.5	タンパク質の C α 原子と水分子の酸素原子の間の RDF	33
3.7	空間分布関数 (spatial_distr, sdf)	34
3.7.1	理論背景	34
3.7.2	オプション一覧	35
3.7.3	出力ファイルのフォーマット	38
3.7.4	解析例	38
3.7.4.1	タンパク質周りの水分子の SDF	38
3.8	平均二乗偏差 (rmsd)	39
3.8.1	理論背景	39
3.8.2	オプション一覧	39
3.8.3	出力ファイルのフォーマット	40
3.8.4	解析例	41
3.8.4.1	水中におけるタンパク質の結晶構造からの RMSD	41
3.9	相互作用エネルギー (interaction_energy, ene)	41
3.9.1	オプション一覧	42
3.9.2	出力ファイルのフォーマット	45
3.9.3	解析例	45
3.9.3.1	溶質間の静電相互作用と LJ 相互作用の和	45
3.9.3.2	溶質間の LJ 相互作用	46
3.10	脂質膜秩序 (lipid_order, scd)	47
3.10.1	オプション一覧	47
3.10.2	出力ファイルのフォーマット	48
3.10.3	解析例	48
3.10.3.1	DPPC 脂質膜の S_{CD} 計算	48
3.11	界面法線方向 (z) に沿った分布関数 (z_profile, zprof)	48
3.11.1	オプション一覧	49
3.11.2	出力ファイルのフォーマット	50
3.11.3	解析例	50
3.11.3.1	脂質膜-水系における水分子の数密度分布	50
3.12	特定の方向 z に対する分子の配向 (z_orient, zori)	51
3.12.1	オプション一覧	52
3.12.2	出力ファイルのフォーマット	54
3.12.3	解析例	54
3.12.3.1	脂質二分子膜内における脂質分子の配向分布	54
3.13	クラスターサイズ (cluster_size)	55
3.13.1	オプション一覧	55
3.13.2	出力ファイルのフォーマット	56
3.13.3	解析例	56
3.13.3.1	水クラスターのサイズ分布	56

4.1	はじめに	57
4.2	共通パラメータ	58
4.2.1	&input_param	59
4.2.2	時系列データのフォーマット	61
4.2.3	統計重みデータのフォーマット	61
4.2.4	&output_param	61
4.3	ヒストグラム (histogram, hist)	62
4.3.1	パラメータ一覧	62
4.3.2	&option_param	62
4.3.3	出力ファイルのフォーマット	63
4.3.4	解析例	63
4.4	平均力ポテンシャル (potential_of_mean_force, pmf)	64
4.4.1	理論背景	64
4.4.2	パラメータ一覧	65
4.4.3	&option_param	65
4.4.4	&matplotlib_param	68
4.4.5	出力ファイルのフォーマット	70
4.4.6	解析例	71
4.5	制限つき動径平均力ポテンシャル (restricted_radial, rrpmf)	71
4.5.1	理論背景	72
4.5.2	パラメータ一覧	73
4.5.3	入力ファイル (時系列データ) のフォーマット	74
4.5.4	&option_param	74
4.5.5	&bootopt_param	76
4.5.6	出力ファイルのフォーマット	77
4.5.7	解析例	77
4.6	フレーム抽出 (extract_frame, extf)	78
4.6.1	パラメータ一覧	78
4.6.2	&option_param	78
4.6.3	出力ファイルのフォーマット	79
4.6.4	解析例	79
4.7	フレームのランダム抽出 (random_frame, randf)	80
4.7.1	パラメータ一覧	80
4.7.2	&option_param	80
4.7.3	出力ファイルのフォーマット	82
4.7.4	解析例	82
4.8	移動平均 (moving_average, movave)	82
4.8.1	パラメータ一覧	83
4.8.2	&option_param	83
4.8.3	出力ファイルのフォーマット	84
4.8.4	解析例	85
4.9	関数の平均 (average_function, avef)	85
4.9.1	パラメータ一覧	86
4.9.2	&option_param	86
4.9.3	&bootopt_param	87

4.9.4	出力ファイルのフォーマット	87
4.9.5	解析例	88
第 5 章	Integral-Equation formalism of Population DYNamics (IEPDYN)	89
5.1	理論背景	89
5.1.1	IEPDYN の理論表式	89
5.1.2	境界条件の導入	90
5.1.3	再帰確率 (Returning probability, RP) 理論との連携	91
5.1.4	$P_j(t)$ の時間積分に対する解析的評価	92
5.1.5	平衡ポピュレーション $P_j(\infty)$ の解析的評価	93
5.2	パラメーター一覧	94
5.3	<code>&option_param</code>	95
5.4	<code>&state</code>	99
5.5	補助ツール	99
5.6	出力ファイル	100
5.7	解析例	101
5.7.1	バイアスポテンシャルなしの MD トラジェクトリを利用する場合 (<code>use_perturbed_traj = .false.</code>)	102
5.7.1.1	リスタートファイル (<code>krhist</code>) の作成	102
5.7.1.2	平衡ポピュレーション値の計算	103
5.7.1.3	$P_B(t)$ の計算	105
5.7.1.4	$P_B(t)$ の時間積分の解析的評価	106
5.7.2	バイアスポテンシャルありの MD トラジェクトリを利用する場合 (<code>use_perturbed_id = .true.</code>)	108
5.7.2.1	リスタートファイル (<code>krhist</code>) の作成	108
5.7.2.2	平衡ポピュレーション値の計算	112
5.7.2.3	$P_B(t)$ の計算	114
5.7.2.4	$P_B(t)$ の時間積分の解析的評価	115

第1章 ANATRA とは？

ANATRA (*Analyze Trajectories*) は分子動力学 (Molecular Dynamics, MD) シミュレーションのトラジェクトリを解析する Tcl/Fortran90 プログラム群である。

系の特定の原子・分子やタンパク質の特定のアミノ酸残基に注目して解析を行うことが多いが、その特定の部位を抜き出してくる汎用プログラムを書くのは容易ではない。というのも、研究レベルでは『残基番号 11 番目から 50 番目までの部分でかつ残基名が ALA でかつ重原子』のような複数の条件を同時に満たす原子群を取り出す必要があるが、これらの条件により生ずる集合の和や積を解釈するプログラムの作成は相当に大変だからである。一方で、世界中の MD ユーザが利用している VMD (*Visual Molecular Dynamics*) では、特定の部位を選択するための優れた機能 (Atomselection) が提供されている。例えば、上記の例を VMD の Atomselection の文法で書くと、

```
resid 11 to 50 and resname ALA and noh
```

であり、直感的にも理解しやすい。そこで **ANATRA** では、VMD の Atomselection を利用したプログラム構成にすることとした。従って、部位選択に関しては、Atomselection と全く同じ文法を使うことが可能であり、VMD ユーザーにとっては新しい部位選択の文法を覚える必要がない。

ANATRA は解析の種類に応じた多数の Tcl/Fortran プログラムから構成されているが、これらを一元的に管理する "anatra" という名前のプログラムを提供した。ユーザーは

```
$ anatra 解析モード -option1 -option2 ...
```

の形で様々な解析を実行できる。このプログラムでは、解析モードに応じた Tcl スクリプトを VMD 上から実行することで Atomselection 機能を用い、選択した部位の情報を Fortran プログラムに入力として与えることで特定の部位に特化した解析を行っているが、ユーザーはそのことを意識する必要はなく、一連の作業を一気に行える。

同時に、Fortran プログラム側では Atomselection に対応する実装がないために、ソースコードの複雑化を回避できており、ユーザー側が **ANATRA** で提供しているライブラリを元に新たな解析プログラムを作成することも容易である。¹

一方で、時系列データからの分布や時間相関関数の計算など、部位選択の機能が必要がない場合も多い。これらについては、Tcl スクリプトを経由せずに機能する Fortran プログラム群を "fanatra" (*Fortran ANATRA*) として一元化したプログラムを提供している。fanatra の場合は

```
$ fanatra 解析モード コントロールファイル
```

の形で解析を実行する。

¹ 将来的には、開発者向けのマニュアルも整備したいと考えている。

- **Project Leader/Main Developer**

- 笠原 健人 (大阪大学基礎工学研究科化学工学領域 松林研究室)

- **Contributor**

- 昌山 廉 (大阪大学基礎工学研究科化学工学領域 松林研究室)
 - 沖田 和也 (大阪大学基礎工学研究科化学工学領域 松林研究室)
 - 松原 優弥 (大阪大学基礎工学研究科化学工学領域 松林研究室)
 - 丸山 優星 (大阪大学基礎工学研究科化学工学領域 松林研究室)
 - 岡部 涼 (大阪大学基礎工学研究科化学工学領域 松林研究室)
 - 山下 湧輝 (大阪大学基礎工学研究科化学工学領域 松林研究室)
 - 佐伯 修都 (大阪大学基礎工学研究科化学工学領域 松林研究室)
 - 松林 伸幸 (大阪大学基礎工学研究科化学工学領域 松林研究室)
-

ANATRA は、**GNU General Public License version 2.0 (GPL v2.0)** の下、配布されている。

ANATRA

Copyright © 2025 Kento Kasahara (Univ. of Osaka)

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

また、下記に示す外部ライブラリに依存して開発が進められており、パッケージ内に同梱されている。これらについては、下記に示すライセンスが適用される。

- **HDF5 (Hierarchical Data Format 5) Software Library and Utilities**

Copyright 2006 by The HDF Group.

NCSA HDF5 (Hierarchical Data Format 5) Software Library and Utilities
Copyright 1998-2006 by The Board of Trustees of the University of Illinois.

All rights reserved.

This software library and utilities is covered by the 3-clause BSD License.

Redistribution and use in source and binary forms, with or without modification, are permitted for any purpose (including commercial purposes) provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions, and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions, and the following disclaimer in the documentation and/or materials provided with the distribution.
3. Neither the name of The HDF Group, the name of the University, nor the name of any Contributor may be used to endorse or promote products derived from this software without specific prior written permission from The HDF Group, the University, or the Contributor, respectively.

DISCLAIMER: THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

For further details, please refer to the full license text available at <https://opensource.org/licenses/bsd-3-clause>

You are under no obligation whatsoever to provide any bug fixes, patches, or upgrades to the features, functionality or performance of the source code (“Enhancements”) to anyone; however, if you choose to make your Enhancements available either publicly, or directly to The HDF Group, without imposing a separate written license agreement for such Enhancements, then you hereby grant the following license: a non-exclusive, royalty-free perpetual license to install, use, modify, prepare derivative works, incorporate into other computer software, distribute, and sublicense such enhancements or derivative works thereof, in binary and source code form.

- **NetCDF**

Copyright 2025 Unidata

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)

ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

- **xdrfile-1.1.4**

Copyright © 2009-2014, Erik Lindahl & David van der Spoel

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

<https://ftp.gromacs.org/pub/contrib/>

- **xdf.F90**

This routine was originally developed by Wes Barnett and distributed under the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

<https://github.com/wesbarnett/libgmxfort>

Modified version of the routine was developed by Kai-Min Tu.

<https://github.com/kmtu/xdrfort>

- **mt19937.f**

This routine was developed by Makoto Matsumoto and Takuji Nishimura and was distributed under the GNU Library General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

<https://www.math.sci.hiroshima-u.ac.jp/m-mat/MT/VERSIONS/FORTRAN/fortran.html>

第2章 ANATRA のインストール

この章では、ANATRA のインストール方法について紹介する。

2.1 ANATRA を利用する上で必要となるプログラム

- VMD (> 1.7.3)
- Intel コンパイラ (Intel OneAPI) または GCC (> 8.5.0)
- bash

ログインシェルとして、bash を用いていることを前提としている。¹

VMD については、

```
https://www.ks.uiuc.edu/Development/Download/download.cgi?PackageName=VMD
```

から無料でダウンロードできる。ただし、アカウント登録が必要である。²

2.2 ソースコードからインストールする

ここでは、自身でソースコードから ANATRA をインストールする方法を紹介する。最新版のソースコードは GitHub 上で管理されている。

```
https://github.com/kenkasa/anatra
```

まず、GitHub リポジトリからソースコードをダウンロードする。ANATRA をインストールしたいディレクトリを /home/USER/software とすると、まずこのディレクトリに移動ののち、git clone を実行する。

```
$ cd /home/USER/software
$ git clone https://github.com/kenkasa/anatra.git
$ ls
```

(次のページに続く)

¹ ただし、次節で述べるように、zsh などの場合についても、自身で rc ファイル (~/.zshrc など) を修正すれば、bash 以外をログインシェルに用いている場合でも利用可能である。

² VMD のインストール方法も簡単に述べておく。web サイトからダウンロードした vmd-XXX.tar.gz を解凍すると、vmd-XXX というディレクトリが生成されるので、そのディレクトリへ移動する。configure ファイルをテキストエディタで開き、ファイル上部にある install_bin_dir=に VMD をインストールしたいディレクトリを入力する。その後、Linux マシンの場合は"./configure LINUXAMD64"と実行した上で、src ディレクトリに移動し、make install を実行すると、install_bin_dir=で指定したディレクトリに vmd プログラムが配置されているはずである。このディレクトリに PATH を通しておけば、インストール完了である。

```

anatra

$ cd anatra
$ ls
README.md
install.sh
bin/
tcl/
f90/
docs/
utility/

```

`install.sh`を実行することで、ANATRA のインストールが開始される。過去に ANATRA のインストールを行ったことがある場合は、`~/.bashrc` または `~/.zshrc` に次の 3 行が含まれているので、実行前に、あらかじめ削除しておく必要がある。ただし、`$ANATRA_PATH` が今回インストールする ANATRA と同じディレクトリを指している場合は、そのままにしても良い (もし分からない場合は下記 3 行を削除すると良い)。

```

export ANATRA_PATH=...
export PATH=$ANATRA_PATH/bin:$PATH
export LD_LIBRARY_PATH=$ANATRA_PATH/f90/lib/external/netcdf/netcdf/lib:$LD_LIBRARY_PATH

```

`install.sh` の実行オプションを下記に記す。

オプション	値	備考
<code>--compiler=<type></code>	<code>gcc, intel</code>	使用するコンパイラを指定する。
<code>--install-hdf5</code>	—	HDF5 ライブラリをインストールする。
<code>--skip-install-lib</code>	—	外部ライブラリのインストールを省略する。
<code>--with-openblas</code>	—	Intel MKL の代わりに OpenBLAS を使用する。事前に環境変数 <code>OPENBLAS_ROOT</code> を設定し、OpenBLAS ディレクトリのインストール先を指定する必要がある。
<code>--with-ffte</code>	—	Intel MKL の FFT の代わりに FFTE ライブラリを使用する。

`install.sh` を実行する際、引数でコンパイラを指定することが出来る。

```
$ ./install.sh --compiler=<gcc or intel>
```

デフォルトは `intel` である。Intel OneAPI が利用できる環境の場合は `intel` を指定することを推奨する。

Intel compiler を利用する場合は、

```
$ ./install.sh --compiler=intel
```

のように実行する。

スクリプトを実行すると、ビルドの進捗がログとして表示される。ビルド終了直前で下記のメッセージが表示されれば、全ての解析プログラムのビルドが正常終了したと考えて良い。

```
Installation of ANATRA fortran programs have been  
succesfully finished!!
```

また、変数として\$ANATRA_PATHが定義されており、anatra コマンドに PATH が通っていれば、インストール成功である。

```
$ echo $ANATRA_PATH  
/home/USER/software/anatra  
  
$ which anatra  
/home/USER/software/anatra/bin/anatra
```

Fortran ソースコードを書き換えて再ビルドする場合、install.sh を上記のオプションで実行すると、外部ライブラリを一からビルドするため、時間がかかる。そこで、オプションとして--skip-install-libを加えると、ライブラリのビルドをスキップできる。ただし、前回のビルドからコンパイラを変更する場合には、外部ライブラリも含めて再ビルドする必要がある。

ANATRA をインストールするマシンの環境によっては要求しているライブラリなどが不足しているために、インストールに失敗することがある。その場合の対応法については、以下を参照されたい。

2.2.1 HDF5 ライブラリを併せてビルドする場合

ANATRA は NetCDF 形式のトラジェクトリをサポートする関係で、HDF5 ライブラリが必要となる。Linux ディストリビューションの多くは、標準で HDF5 ライブラリをプリインストールしている。もし、インストールされていない、またはインストールされているか分からない場合には、--install-hdf5 を下記のように加えれば、ANATRA に同梱されている HDF5 をインストールする。

```
$ ./install.sh --compiler=<gcc or intel> --install-hdf5
```

2.2.2 OpenBLAS を利用する場合

install.sh は、デフォルトでは Intel OneAPI に同梱されている数学ライブラリ Intel Math Kernel Library (MKL) を利用してコンパイルするようになっているが、Intel OneAPI をインストールしていない場合でも、OpenBLAS で代替することが可能である。

OpenBLAS は GitHub 上からダウンロードすることが可能である。

```
git clone https://github.com/OpenMathLib/OpenBLAS.git
```

上記コマンドを実行すると、OpenBLAS というディレクトリにソースコードが格納される。そのディレクトリ内に入って、`make` と `make install` を実行すれば良い。OpenBLAS ディレクトリ上にライブラリ等をインストールする場合を下記に記す。

```
cd OpenBLAS
make
make PREFIX=/path/to/OpenBLAS install
```

インストール後に、グローバル変数として `OPENBLAS_ROOT` を `~/.bashrc` や `~/.zshrc` の中に下記のように定義しておく。

```
export OPENBLAS_ROOT=/path/to/OpenBLAS
```

その上で、ANATRA の `install.sh` を `--with-openblas` のオプションをつけて実行すれば良い。

```
./install --compiler=gcc --with-openblas
```

なお、OpenBLAS を利用する数学ライブラリとして指定すると、自動的に高速フーリエ変換 (Fast Fourier Transform, FFT) のライブラリとして `ffte` が利用されるようになる。

第3章 anatra の使い方

3.1 はじめに

anatra は VMD と Fortran を用いて解析を行うプログラムである。この章では、**anatra** の使い方を解説する。

先にも述べたが、基本的にこのプログラムを用いる解析では、まず VMD を用いた分子の選択などの処理を通して中間ファイルを生成、それを Fortran にて解析するという二段階の処理を踏んでいる。

利用可能な解析は"anatra -h"を実行することで表示される。

```
$ anatra -h
Available analysis in ANATRA
trjconv          (trj)   : Convert trajectory
distance         (dis)   : Distance Analysis
center_of_mass   (com)   : Center-of-Mass Analysis
radial_distr     (rdf)   : Radial-Distribution Analysis
spatial_distr    (sdf)   : Spatial-Distribution Analysis
rmsd             (rmsd)  : Root-Mean-Square-Deviation Analysis
interaction_energy (ene)  : Interaction Energy Analysis
lipid_order      (scd)   : Lipid S_CD order-parameter Analysis
area_per_lipid   (apl)   : Area Per Lipid Analysis
z_profile        (zprof) : Z-profile Analysis
z_orient         (zori)  : Orientation Analysis along z
rotation         (rot)   : Rotation Analysis
cluster_size     (cls)   : Cluster-size Analysis
free_volume      (fvol)  : Free-Volume Analysis
species_info     (spec)  : Get Species information
gauqm_prepper    (gau)   : Gaussian QM input prepper
```

trjconv などが解析モードであり、

```
$ anatra trjconv -option1 ...
```

のように実行する。また括弧内に示されているのはそれぞれの解析モード名の省略版であり、例えば trjconv は、

```
$ anatra trj -option1 ...
```

と実行しても良い。

各解析モードでのオプションなどは"anatra 解析モード -h"で簡単な説明とサンプルが表示される。このヘルプメッセージだけで大体の使い方が分かるようになっているが、より詳細な説明が必要な場合にこのマニュアルを参照すると良いだろう。以下では、trjconv の場合を例として示しておく (VMD のデフォルト出力は省略している)。

```
$ anatra trjconv -h
=====

                          Trajectory Convert

=====

Usage:
anatra trjconv                                \
  -stype      <structure file type>           \
  -sfile      <structure file name>           \
  -tintype    <input trajectory file type>     \
  -tin        <input trajectory file name>     \
  -totype     <output trajectory file type>    \
  -to         <output trajectory file name>    \
  -beg        <first frame to be read> (default: 1) \
  -end        <last frame to be read>         \
              (default: 0, corresponding to last frame)) \
  -selX       <X-th VMD selection> (X=0,1,2...) \
  -fit        <fit is performed or not (true or false)> \
  -centering  <centering is performed or not (true or false)> \
              (default: false) \
  -wrap       <wrap is performed or not (true or false)> \
              (default: false) \
  -wrapcenter <wrapping center (origin or com) (default: fragment)> \
  -wrapcomp   <how to wrap molecules (residue, segid, chain, or fragment)> \
              (residue or segid or chain or fragment or none) \
              (default: fragment) \
  -refpdb     <reference pdb file name>        \
              (necessary if fit = true) \
  -outselid   <selection id for output molecules> \
  -fitselid   <selection id for fitting or centering> \
  -refselid   <selection id for reference>

Remark:
o If you specify fit = true & wrap = true, wrapcomp is automatically changed to 'com'

Example (fitting):
anatra trjconv                                \
  -stype      parm7                            \
```

(次のページに続く)

(前のページからの続き)

```

-sfile      str.prmtop      \
-tintype    dcd             \
-tin        inp.dcd        \
-totype     dcd            \
-to         out.dcd        \
-beg        1              \
-end        150            \
-sel0       not water      \
-sel1       resid 1 to 275 and name CA \
-sel2       resid 1 to 275 and name CA \
-fit        true           \
-centering  false          \
-wrap       true           \
-wrapcenter origin        \
-wrapcomp   fragment      \
-refpdb     ref.pdb        \
-outselid   0              \
-fitselid   1              \
-refselid   2              \

```

上記の例が示すように、**anatra** では多くのオプションを含んでいるため、**bash** スクリプトなどの中で実行すると良い。以下にスクリプト例を示しておく。

```
#!/bin/bash
```

```

anatra trjconv      \
  -stype    parm7    \
  -sfile    str.prmtop \
  -tintype   dcd      \
  -tin       INP.dcd  \
  -totype    dcd      \
  -to        OUT.dcd  \
  -sel0      resid 1 to 20 \
  -outselid  0

```


3.2 共通オプション

anatra には多くの解析モードが存在するが、解析モード間で多くの共有オプションが存在する。まず、それらを一覧として以下に示しておく。その後に、各オプションの内容について解説する。

オプション名	機能	例
-stype	構造ファイルの種類を指定	-sfile parm7
-sfile	構造ファイル名を指定	-stype A.prmtop
-tintype	入力トラジェクトリの種類を指定	-tintype dcd
-tin	入力トラジェクトリ名を指定	-tin in.dcd
-flist_traj	入力トラジェクトリ一覧を記載したファイルを指定	-flist_traj list.txt
-totype	出力トラジェクトリの種類を指定	-totype dcd
-to	出力トラジェクトリ名を指定	-to out.dcd
-selx ($x = 0, 1, \dots$)	選択する部位を指定 (x を selid と呼ぶ)	-sel0 resid 1 to 20
-mode or -modex ($x = 0, 1, \dots$)	選択した部位をどのように分割するかを指定	-mode residue
-fhead	解析結果の出力ファイル名のヘッダー	-fhead out
-prep_only	解析の前処理のみを行うか指定	-prep_only true

• -stype <構造ファイルの種類>

- Default: N/A
- Example: **-stype parm7**

構造ファイル (structure file) を指定する。構造ファイルは原子種情報 (や原子間の結合情報) を格納しているファイルのことを指す。多くのタイプのトラジェクトリファイル (**dcd** や **xtc** など) では原子の座標情報のみが格納されており、解析を進めるためには構造ファイルが必要となる。対応している構造ファイルは、おおよそ VMD がサポートしている全ての構造ファイルの種類を指定することが可能である。代表的なものを以下に示す。

- **pdb**: PDB ファイル
- **gro**: GRO ファイル
- **psf**: CHARMM PSF ファイル
- **parm7**: Amber パラメータファイル (多くの場合拡張子は **.prmtop**)

最も完全な情報を含んでいるのは、CHARMM の場合には **psf**、Amber の場合には **parm7** なので、できる限り、これらのファイルを指定しておくが良い。

• -sfile <構造ファイル>

- Default: N/A

- Example: `-sfile complex.prmtop`

構造ファイルの名前を指定する.

- **-tintype <入力トラジェクトリファイルの種類>**

- Default: `dcd`
- Example: `-tintype dcd`

入力トラジェクトリファイルの種類を指定する. 現在, `trjconv` を除いて, 指定できるのは `dcd` と `xtc` と `netcdf` に限定されている. `trjconv` では, VMD がサポートする全ての種類のトラジェクトリが指定可能である.

- **-tin <入力トラジェクトリファイル>**

- Default: `N/A`
- Example: `-tin INP.dcd`

入力トラジェクトリのファイル名を指定する. 複数のトラジェクトリファイルを読み込ませる場合は, “`-tin run1.dcd run2.dcd`”のように並べて書く. またワイルドカード (*) を用いることも可能であり, “`-tin run*.dcd`”のように書く. 次に示す`-flist_traj` を用いて入力のトラジェクトリを指定することも可能である.

- **-flist_traj <テキストファイル>**

- Default: `N/A`
- Example: `-flist_traj trajlist.txt`

入力トラジェクトリの一覧を記したテキストファイルを指定する. `-tin` と `-flist_traj` の両方が指定された場合には, `-flist_traj` が優先される. 入力テキストファイルのフォーマットについては, 次に示すように, 入力のトラジェクトリ名が一行ずつ記されている必要がある.

```
traj1.dcd
traj2.dcd
traj3.dcd
```

- **-totype <出力トラジェクトリファイルの種類>**

- Default: `dcd`
- Example: `-totype dcd`

出力トラジェクトリファイルの種類を指定する. 現在, `trjconv` を除いて, 指定できるのは `dcd` と `xtc` に限定されている. `trjconv` では, VMD がサポートする全ての種類のトラジェクトリが指定可能である.

- **-to <出力トラジェクトリファイル>**

- Default: N/A
- Example: `-to OUT.dcd`

出力トラジェクトリのファイル名を指定する.

- **-sel x <部位選択>** ($x = 0, 1, \dots$)

- Default: N/A
- Example: `-sel0 resid 1 to 20 and name CA`

選択する部位を VMD の Atomselection の文法に従って指定する. `-sel $x$` の x のことを `selid` と呼ぶ. 多くの解析モードでは, `-sel $x$` で指定された部位の解析を行う. また, 解析モードによっては, どの `selid` を解析に使うかを明示的に指定することも多い. (例えば, `trjconv` の `-outselid` など)

- **-mode x <residue or whole or atom>** ($x = 0, 1, \dots$)

- Default: 解析モードに依存
- Example: `-mode0 residue`

`-sel $x$` で指定した部位をどのように分割して分子を定義するかを指定する (図 3.1). 解析によっては, `-sel0` の 1 つしか必要がない場合もあり, その解析では `-mode` で指定し, 複数の部位選択が必要な場合には, `-mode0`, `-mode1` のようにそれぞれで対応する分割スキームを指定する. どちらのオプションで指定するかは各解析モードでの解説にて述べる.

- **residue**: 選択部位を残基ごとに分割して分子と定義する.
- **whole**: 選択部位全体を一つの分子と定義する.
- **atom**: 選択部位の構成分子一つ一つを分子と定義する.

例えば, `-sel0 resname LIG` で複数のリガンド分子が選択されたとして, それぞれのリガンドの重心を知りたい場合には, `-mode residue` と設定する必要がある. 逆に, `-sel0 protein` と選択して, タンパク質一つを丸ごと解析対象としたい場合 (タンパク質の重心の時間変化など), `"-mode whole"` とする必要がある.

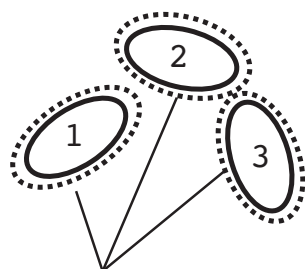
- **-fhead <出力ファイル名のヘッダー>**

- Default: 解析モードに依存
- Example: `-fhead out`

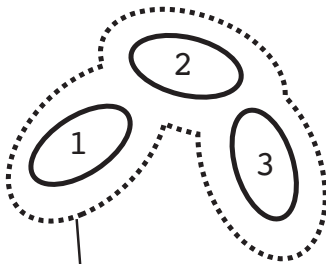
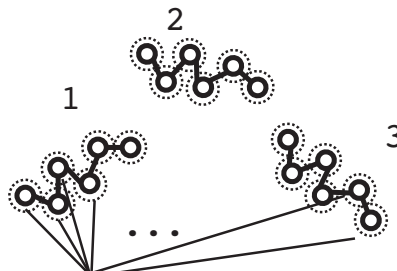
解析の出力結果を格納するファイル名のヘッダーを指定する. 例えば, `-fhead out` と指定すると, `out.XXX`, `out.YYY` などのファイルが出力される.

- **-prep_only**

- Default: false

`-mode residue`

residue 毎に解析

`-mode whole`複数の residue を
ひとまとめにして解析`-mode atom`

原子毎に解析

図 3.1: `-mode` の使い分け. `residue` では, 選択した部位を残基ごとに分割して分子と定義する. `whole` では, 選択した部位をひとかたまりの分子として定義する. `atom` では, 選択した部位を構成原子ごとに分割して分子と定義する.

– Example: `-prep_only true`

実際の解析まで行わず, それに必要な前処理 (パラメータファイルの作成) までで止めるかを指定する.

3.3 トラジェクトリ変換 (trjconv)

`trjconv` は MD 計算で得られたトラジェクトリに対して, 種々の変換を行う解析モードである. 主な用途として次のようなものが挙げられる.

- トラジェクトリのファイル形式を変更したい場合
- トラジェクトリ内の特定の部位のみを含むトラジェクトリを作成したい場合
(例. トラジェクトリ内の水を除いたトラジェクトリの作成)
- ラップされていないトラジェクトリを周期境界条件に基づいてラップしたい場合 (PBC ラップ)
- トラジェクトリ内の特定の部位を参照構造と重なるように系全体を並進回転させたい場合 (フィッティング)
- トラジェクトリ内の特定の部位の位置が座標原点となるように系全体を並進させたい場合 (センタリング)

3.3.1 オプション一覧

オプション名	備考	オプション名	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-totype	Common	-to	Common
-sel x	Common	-beg	Mode specific
-end	Mode specific	-fit	Mode specific
-centering	Mode specific	-wrap	Mode specific
-wrapcenter	Mode specific	-wrapcomp	Mode specific
-refpdb	Mode specific	-outselid	Mode specific
-fitselid	Mode specific	-refselid	Mode specific

- **-beg <トラジェクトリの読み込み開始ステップ>**

- Default: 1
- Example: `-beg 10`

トラジェクトリの読み込み開始ステップを指定する。ただし、ステップ数は1始まりでカウントしていることに注意されたい。上記の例は、10 ステップ目からトラジェクトリの読み込みを開始することを意味している。

- **-end <トラジェクトリの読み込み終了ステップ>**

- Default: 0 (トラジェクトリの最終ステップに対応)
- Example: `-end 100`

トラジェクトリの読み込み終了ステップを指定する。ただし、ステップ数は1始まりでカウントしていることに注意されたい。上記の例は、100 ステップ目までトラジェクトリを読み込むことを意味している。

- **-fit <true or false>**

- Default: false
- Example: `-fit true`
- Note: true の場合は、`-fitselid`、`-refpdb`、`-refselid` の指定が必要となる。

フィッティングを行うかどうかを指定する。フィッティングとは、トラジェクトリの特定部位を参照構造に重なるように系全体の並進・回転を行う操作のことを意味する。利用する際、トラジェクトリ内のフィッティング部位に対応する selid (`-sel $x$` の x のこと) を指定する `-fitselid`、参照構造の PDB ファイルを指定する `-refpdb`、参照構造内のフィッティングに使う部位の selid を指定する `-refselid` が必要となる。

- **-wrap <true or false>**

- Default: false
- Example: `-wrap true -wrapcomp fragment -wrapcenter origin`
- Note: ラップを行うときの中心を座標原点から変更したいときには、`-wrapcenter` を利用する。どのようなユニットを 1 つの分子として扱ってラップするかは `-wrapcomp` で指定する。

アンラップされた状態のトラジェクトリをラップするかどうかを指定する。ラップするときの中心は、デフォルトで座標原点 $(x, y, z) = (0, 0, 0)$ に設定されているが、`-wrapcenter` オプションを利用することで特定部位の重心に変更することが可能である。

- **-centering <true or false>**

- Default: false
- Example: `-centering true -sel1 resid 1 to 200 -fitselid 1`
- Note: 中心に並進移動させる部位の `selid` は `-fitselid` で指定する。

センタリングを行うかどうかを指定する。センタリングとは、トラジェクトリの特定の部位 (の重心) が座標原点に配置されるように系全体を並進移動させる操作のことを指す。どの部分を中心に移動させるかは `-fitselid` で指定する。

- **-refpdb <PDB file>**

- Default: N/A
- Example: `-refpdb A.pdb -sel2 resid 1 to 200 -refselid 2`
- Note: 参照構造内のフィッティングに使う部位の `selid` を `-refselid` で指定する。

フィッティングを行う際の、参照構造の PDB ファイル名を指定する。従って、このオプションは `-fit true` のときに必須となる。参照構造内のフィッティングに使う部位は `-refselid` で指定する。

- **-outselid <selid>**

- Default: N/A
- Example: `-sel0 resid 1 to 200 -outselid 0`

トラジェクトリとして出力する部位に対応する `selid` を指定する。

- **-fitselid <selid>**

- Default: N/A
 - Example: `-sel1 resid 1 to 200 -fitselid 1`
-

フィッティングに使うトラジェクトリ内の部位に対応する `selid` を指定する。

- **-wrapcenter <origin or com>**

- Default: `origin`
- Example: `-wrapcenter origin`
- Note: `com` の場合は中心に据える部位の `selid` を `-fitselid` で指定する。

ラップする際の系の中心をどこに据えるかを指定する。

- **origin**: 系の中心として座標原点を指定する。
 - **com**: 系の中心として特定の部位を指定する。中心に据える部位の `selid` を `-fitselid` で指定する。フィッティングまたはセンタリングを同時に行う場合、強制的に `com` が指定される。
-

- **-wrapcomp <fragment or residue or nocomp>**

- Default: `fragment`
- Example: `-wrapcomp fragment`

ラップを実行する際、どのようなユニットで分子を区切るかを指定する。例えば、シミュレーションボックスをはみ出した原子が存在する場合にその原子のみを PBC に基づいて反対側に移動させるようにするか、もしくは、分子全体がボックスからはみ出した場合にのみ反対側に移動させるかを指定する。その際、ユーザー側はどのような原子団 (ユニット) を分子として定義するかを指定する必要がある。

- **fragment**: 原子 i と j が他の原子との結合を介して繋がっている場合、それらは同じユニットに属していると考え、このような原子のセットを一つの分子としてラップが実行される。
 - **residue**: 一つの残基番号に属する原子群を一つの分子としてラップが実行される。
 - **nocomp**: 原子 1 つ 1 つを分子として扱ってラップが実行される。
-

3.3.2 解析例

ここでは、よく行う解析について、最低限のオプションを使ったスクリプトの例を示しておく。従って、`-beg`, `-end` などのオプションは使用しない。

3.3.2.1 トラジェクトリから水を除去

```
#!/bin/bash
```

```
anatra trjconv \
  -stype      parm7 \
  -sfile      complex.prmtop \
  -tintype    dcd \
  -tin        INP.dcd \
  -totype     dcd \
  -to         OUT.dcd \
  -sel0       not water \
  -outselid   0
```

3.3.2.2 PBC ラップ

```
#!/bin/bash
```

```
anatra trjconv \
  -stype      parm7 \
  -sfile      complex.prmtop \
  -tintype    dcd \
  -tin        INP.dcd \
  -totype     dcd \
  -to         OUT.dcd \
  -sel0       all \
  -wrap       true \
  -wrapcenter origin \
  -wrapcomp   fragment \
  -outselid   0
```

3.3.2.3 フィッティング

ここでは、水溶液中にタンパク質 (resid 1 to 192 とする) が一つ存在する系を考える。トラジェクトリ中のタンパク質の C α 原子を参照構造 PDB の C α 原子にフィットさせる場合の例を示す。

```
anatra trjconv \
  -stype      parm7 \
  -sfile      complex.prmtop \
  -tintype    dcd \
  -tin        INP.dcd \
```

(次のページに続く)

(前のページからの続き)

```

-totype      dcd          \
-to          OUT.dcd      \
-refpdb      REF.pdb      \
-sel0        all          \
-sel1        resid 1 to 192 and name CA \
-sel2        resid 1 to 192 and name CA \
-fit         true         \
-outsclid    0            \
-fitsclid    1            \
-refscld     2            \

```

3.3.2.4 センタリング

ここでは、水溶液中にタンパク質 (resid 1 to 192 とする) が一つ存在する系を考える。トラジェクトリ中のタンパク質の重心を座標原点に配置させるセンタリングを行う場合の例を示す。

```

anatra trjconv          \
  -stype      parm7      \
  -sfile      complex.prmtop \
  -tintype     dcd        \
  -tin        INP.dcd     \
  -totype     dcd        \
  -to         OUT.dcd     \
  -sel0       all         \
  -sel1       resid 1 to 192 \
  -centering  true        \
  -outsclid   0           \
  -fitsclid   1           \

```

3.3.2.5 PBC ラップ&フィッティング

ここでは、水溶液中にタンパク質 (resid 1 to 192 とする) が一つ存在する系を考える。トラジェクトリ中のタンパク質の C α 原子を参照構造 PDB の C α 原子にフィットさせるのと同時に、PBC ラップを実行する例を示す。この際、ラップ中心は C α 原子の重心である。

```

anatra trjconv          \
  -stype      parm7      \
  -sfile      complex.prmtop \
  -tintype     dcd        \
  -tin        INP.dcd     \

```

(次のページに続く)

(前のページからの続き)

```

-totype      dcd                \
-to          OUT.dcd            \
-refpdb      REF.pdb           \
-sel0        all                \
-sel1        resid 1 to 192 and name CA \
-sel2        resid 1 to 192 and name CA \
-fit         true               \
-wrap        true               \
-wrapcenter  com                \
-wrapcomp    fragment          \
-outselid    0                  \
-fitselid    1                  \
-refselid    2

```

3.3.2.6 PBC ラップ&センタリング

中心に据える部分として、脂質膜 (resid 1 to 200 and segid MEMB とする) を設定した場合の例を示す。

```

anatra trjconv                \
  -stype      psf              \
  -sfile      complex.psf      \
  -tintype    dcd              \
  -tin        INP.dcd          \
  -totype     dcd              \
  -to         OUT.dcd          \
  -sel0       all              \
  -sel1       resid 1 to 200 and segid MEMB \
  -centering  false            \
  -wrap       true             \
  -wrapcenter com              \
  -wrapcomp   fragment         \
  -outselid   0                \
  -fitselid   1

```

3.4 距離 (distance, dis)

`distance` は選択した部位間の距離を計算する解析モードである。通常の距離計算の場合、選択した部位の分子重心間の距離が計算される。また、部位間の最近接距離 (minimum distance) 計算なども可能である。

3.4.1 オプション一覧

オプション名	備考	オプション名	備考
<code>-stype</code>	Common	<code>-sfile</code>	Common
<code>-tintype</code>	Common	<code>-tin</code>	Common
<code>-flist_traj</code>	Common	<code>-fhead</code>	Common
<code>-sel0</code>	Common	<code>-sel1</code>	Common
<code>-mode0</code>	Common	<code>-mode1</code>	Common
<code>-pbc</code>	Mode specific	<code>-dt</code>	Mode specific
<code>-distance_type[†]</code>	Mode specific	<code>-mindist_type0[†]</code>	Mode specific
<code>-mindist_type1[†]</code>	Mode specific	<code>-prep_only</code>	Common

通常の距離を計算する上で不要なオプションには[†]を付けているので、通常の距離を求めるだけであれば、それらのオプションは無視して良い。

- `-sel0` <部位選択>
- `-sel1` <部位選択>
 - Default: N/A
 - Example: `-sel0 resid 1 to 20 and noh -sel1 resname LIG`

この解析モードでは基本的に選択した二つの部位 (`selid=1, 2`) の間の距離を計算する。その対となる部位を `-sel $x$` ($x = 1, 2$) で指定する。

- `-mode0` <residue or whole or atom>
 - `-mode1` <residue or whole or atom>
 - Default: residue
 - Example: `-mode0 whole -mode1 residue`
- `-sel0`, `-sel1` で選択した部位をどのように分割するかを指定する。

- `-pbc` <true or false>

- Default: `false`
- Example: `-pbc true`

距離を計算する際に、PBC を考慮するかどうかを指定する。

• **-dt <時間刻み>**

- Default: `1.0`
- Example: `-dt 0.1`

時系列データを出力するときの時間軸 (第 1 列) の時間刻みを指定する。

• **-distance_type <standard or minimum or intra>[†]**

- Default: `standard`
- Example: `-distance_type standard`

距離の種類を指定する。

- **standard**: 通常の距離計算を実行する。
- **minimum**: 部位間の最近接の原子間距離を計算する。
- **intra**: 分子内距離を計算する。

intra は他の 2 つに比べて少し複雑であるので、具体例を交えて使い道を説明する。いま、同種の高分子 (残基名 PLM) が 100 本存在する系を考える。また、両末端の原子の名前を CP1, CP2 とする。このような系で、それぞれの高分子の末端-末端間距離を計算することはよくある。しかし、`-distance_type standard` を選び、`-sel0 name CP1`, `-sel1 CP2` と指定すると、異なる高分子間の CP1-CP2 距離も計算してしまうことになる。`-distance_type intra` では、同一の分子内での距離のみを計算するようになっている。

• **-mindist_type1 <site or com>[†]**

- Default: `site`
- Example: `-mindist_type0 com -mindist_type1 site`

部位内の最近接サイト候補を指定する。

- **site**: 部位を構成する原子を最近接サイト候補に設定する。
- **com**: 部位の重心のみを最近接サイト候補に設定する。

部位 0, 1 がともに **com** の場合、通常の距離計算と等価になる。

3.4.2 出力ファイルのフォーマット

- *.dis ファイル

このファイルには、選択部位間の距離または最近接距離が出力される。-mode0, -mode1 の指定により、部位 0, 1 がそれぞれ n_1 個, n_2 個の分子に分割されたとすると、合計で $n_1 n_2$ 個の距離が出力されることになる。そのときの各データは次のように出力される。

- 1 列目: 時間軸
- 2 列目: 分子対 (1, 1) の距離
- 3 列目: 分子対 (1, 2) の距離
- ...
- n_2 列目: 分子対 (1, n_2) の距離
- $n_2 + 1$ 列目: 分子対 (2, 1) の距離
- $n_2 + 2$ 列目: 分子対 (2, 2) の距離
- ...

距離は、Å 単位で出力される。

3.4.3 解析例

3.4.3.1 通常の原子間距離の計算

タンパク質 (resid 1 to 192) とリガンド 10 分子 (resname LIG) の間の距離を計算する場合を示す。

```
#!/bin/bash

anatra distance \
  -stype      parm7 \
  -sfile      complex.prmtop \
  -tintype    dcd \
  -tin        INP.dcd \
  -fhead      out \
  -pbc        true \
  -distance_type standard \
  -dt         0.1 \
  -sel0       resid 1 to 192 \
  -sel1       resname LIG \
  -mode0      whole \
  -mode1      residue
```

3.5 重心解析・平均二乗変位 (center_of_mass, com)

`center_of_mass (com)` は分子の重心の時間変化や平均二乗変位 (Mean square displacement, MSD) を求めるプログラムである。また、脂質膜内部での分子の 2 次元的拡散 (側方拡散) に対応するため、2 次元 (xy 平面) での MSD 計算も可能である。

3.5.1 理論背景

3.5.1.1 平均二乗変位

平均二乗変位 (MSD) $\chi(t)$ の定義を導入すると同時に、それを求める意義について、拡散方程式に基づいて述べる。

分子の時刻 t での位置を $\mathbf{r}(t)$ とすると、MSD は次式で定義される。

$$\chi(t) = \langle |\mathbf{r}(t) - \mathbf{r}(0)|^2 \rangle. \quad (3.1)$$

つまり $\chi(t)$ は、時間が t だけたった後における分子の移動距離の 2 乗平均を表す。MSD が拡散方程式に基づくと、どのように表されるのかを次に考えてみる。分子の確率密度場を $\rho(\mathbf{r}, t)$ 、拡散係数を D とすると、拡散方程式は次式で表される。

$$\frac{\partial}{\partial t} \rho(\mathbf{r}, t) = D \nabla^2 \rho(\mathbf{r}, t). \quad (3.2)$$

時刻 $t = 0$ で分子は位置 $\mathbf{0}$ に存在していたとすると、初期条件は

$$\rho(\mathbf{r}, t = 0) = \delta(\mathbf{r}), \quad (3.3)$$

となる。この条件の下、拡散方程式を解くと、次式が得られる。

$$\rho(\mathbf{r}, t) = \frac{1}{(4\pi Dt)^{3/2}} \exp \left[-\frac{|\mathbf{r}|^2}{4Dt} \right]. \quad (3.4)$$

この解を用いると、MSD は

$$\chi(t) = \int d\mathbf{r} |\mathbf{r}|^2 \rho(\mathbf{r}, t) \quad (3.5)$$

$$= 6Dt, \quad (3.6)$$

と表される。つまり、MSD は傾き $6D$ で時間に比例して増加していくことになる。ただし、拡散方程式は長時間極限において成り立つ関係式なので、

$$D = \frac{1}{6} \lim_{t \rightarrow \infty} \frac{\langle |\mathbf{r}(t) - \mathbf{r}(0)|^2 \rangle}{t}, \quad (3.7)$$

とすべきである。上式は Einstein の関係式と呼ばれる。この式は、MSD の長時間領域の傾きから拡散係数が求められることを意味している。

3.5.2 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-sel0	Common	-mode	Common
-outmsd	Mode specific	-unwarp	Mode specific
-msddim	Mode specific	-dt	Mode specific
-t_sparse	Mode specific	-t_range	Mode specific
-prep_only	Common		

- **-outmsd <true or false>**

- Default: false
- Example: `-outmsd true`
- Note: true の場合は `-msddim` と `-msdrange` も指定する必要がある。

MSD を計算するかどうかを指定する。

- **-unwrap <true or false>**

- Default: false
- Example: `-unwrap false`

ラップされているトラジェクトリをアンラップするかどうかを指定する。ラップされているトラジェクトリでは MSD が正しく計算できないため、true にする必要がある。ただし、GROMACS のように、分子が PBC によりぶつ切りになっているトラジェクトリには対応できないので、注意されたい。その場合はあらかじめ、GROMACS の `trjconv` にて修正しておく必要がある。

- **-msddim <2 or 3>**

- Default: 3
- Example: `-msddim 3`

MSD の次元を指定する。通常の拡散では 3 を指定するが、脂質膜系での側方拡散を調べる場合には 2 を指定する。

- **-dt <時間刻み>**

- Default: 0.1
- Example: -dt 0.1

スナップショット間の時間刻みを指定する.

- **-t_range <MSD を計算する時間スケール>**

- Default: 10
- Example: -t_range 10

MSD を計算する時間スケールを指定する.

- **-t_sparse <MSD を出力するときの時間刻み>**

MSD を出力するときの時間刻みを指定する. 例えば, -dt 0.1 ($= \delta t$), -t_sparse 1 ($= \Delta t$) のとき,

$$\Delta T = \text{nint} \left(\frac{\Delta t}{\delta t} \right) \delta t \quad (3.8)$$

の時間刻みで MSD ($\text{\AA}^2$) が計算される.

3.5.3 出力ファイルのフォーマット

- ***.com**

このファイルには, 選択部位内の分子それぞれの重心の座標が xyz 形式で出力される.

- ***.msd**

このファイルには, 分子ごとの MSD が出力されている. "-outmsd true"の場合に出力される.

- 1 列目: 時間軸
- 2 列目: 分子 1 の MSD
- 3 列目: 分子 2 の MSD
- ...

- ***.msdave**

このファイルには, 全分子に対して平均をとった MSD が出力されている. "-outmsd true"の場合に出力される.

- 1 列目: 時間軸
- 2 列目: MSD
- 3 列目: MSD の標準偏差

3.5.4 解析例

3.5.4.1 タンパク質の MSD 計算

水溶液中にタンパク質 (resid 1 to 192) が 1 つ溶解している系において、タンパク質の MSD を計算する例を以下に示す。タンパク質は複数のアミノ酸残基から構成されているので、"-mode whole"を指定する必要がある。

```
#!/bin/bash

anatra center_of_mass \
  -stype    parm7 \
  -sfile    complex.prmtop \
  -tintype  dcd \
  -tin      INP.dcd \
  -fhead    out \
  -outmsd   true \
  -sel0     resid 1 to 192 \
  -mode     whole \
  -msddim   3 \
  -dt       0.01 \
  -t_range  100 \
  -t_sparse 0.1
```

3.5.4.2 複数個存在する小分子の MSD 計算

水溶液中にリガンド小分子 (resname LIG) が 100 分子存在する系を考える。分子ごとの MSD とその平均を計算する例を以下に示す。この場合は残基番号によって分子の切れ目を定めることができるので、"-mode residue"を指定する。

```
#!/bin/bash

anatra center_of_mass \
  -stype    parm7 \
  -sfile    complex.prmtop \
  -tintype  dcd \
  -tin      INP.dcd \
  -fhead    out \
  -outmsd   true \
  -sel0     resname LIG \
  -mode     residue \
  -msddim   3 \
  -dt       0.01
```

(次のページに続く)

(前のページからの続き)

```
-t_range    100      \
-t_sparse   0.01
```

3.6 動径分布関数 (radial_distr, rdf)

3.6.1 理論背景

ここでは、動径分布関数の定義について解説する。

3.6.1.1 同種粒子間の動径分布関数

粒子 a が N_a 個存在している系 (体積 V ，粒子 a の平均 (バルク) 数密度 $\rho_a = N_a/V$) を考え、局所密度場 $\hat{\rho}_a(\mathbf{r})$ を次式で定義する。

$$\hat{\rho}_a(\mathbf{r}) = \sum_{i=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i). \quad (3.9)$$

ここで、 $\mathbf{r}_i$ は i 番目の粒子 a の位置である。もし、系に外場などが存在しない場合、局所密度場のアンサンブル平均は平均数密度 ρ_a に等しくなる。つまり、

$$\langle \hat{\rho}_a(\mathbf{r}) \rangle = \rho_a, \quad (3.10)$$

である。次に、2 体密度相関関数を次式で定義する。

$$\chi(\mathbf{r}, \mathbf{r}') = \langle \hat{\rho}_a(\mathbf{r}) \hat{\rho}_a(\mathbf{r}') \rangle \quad (3.11)$$

$$= \left\langle \sum_{i=1}^{N_a} \sum_{j=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_j) \right\rangle. \quad (3.12)$$

上式において、 $i = j$ の項と $i \neq j$ の分けて書くと、

$$\chi(\mathbf{r}, \mathbf{r}') = \sum_{i=1}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_i) \rangle + \sum_{i=1}^{N_a} \sum_{j \neq i}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_j) \rangle, \quad (3.13)$$

第 1 項目は同一粒子が $\mathbf{r}$ と $\mathbf{r}'$ の位置に存在する確率を表しており、これは $\mathbf{r} = \mathbf{r}'$ のとき以外は明らかにゼロとなる。つまり、

$$\sum_{i=1}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_i) \rangle = \rho_a \delta(\mathbf{r} - \mathbf{r}'), \quad (3.14)$$

である。次に、第 2 項は i 番目の粒子が $\mathbf{r}$ の位置に存在し、 j 番目の粒子が $\mathbf{r}'$ の位置に存在する確率を表している。この項は 2 体密度と呼ばれており、2 粒子の相関関係を表している。2 体密度を $\rho_{aa}^{(2)}(\mathbf{r}, \mathbf{r}')$ と表しておくことにする。

$$\rho_{aa}^{(2)}(\mathbf{r}, \mathbf{r}') = \sum_{i=1}^{N_a} \sum_{j \neq i}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_j) \rangle. \quad (3.15)$$

もし、 $\mathbf{r}$ と $\mathbf{r}'$ の位置が互いに遠く離れている場合、そのような配置での 2 粒子間の相関は無視できる。

$$\rho_{aa}^{(2)}(\mathbf{r}, \mathbf{r}') \approx \sum_{i=1}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \rangle \sum_{j \neq i}^{N_b} \langle \delta(\mathbf{r}' - \mathbf{r}_j) \rangle \quad (3.16)$$

$$= \rho_a^2 \left(1 - \frac{1}{N_a}\right). \quad (3.17)$$

無相関の状態を基準にとって、2 体分布関数を次式で定義することにしておく。

$$g_{aa}^{(2)}(\mathbf{r}, \mathbf{r}') = \frac{1}{\rho_a^2 (1 - 1/N_a)} \rho_{aa}^{(2)}(\mathbf{r}, \mathbf{r}'). \quad (3.18)$$

もし、粒子を十分に多く含んでいる系を取り扱っている場合は、 $1/N_a \simeq 0$ と考えて、

$$g_{aa}^{(2)}(\mathbf{r}, \mathbf{r}') = \frac{1}{\rho_a^2} \rho_{aa}^{(2)}(\mathbf{r}, \mathbf{r}'), \quad (3.19)$$

としても良い。

系が均一の場合 (外場が存在しない場合など)、2 体分布関数 (および 2 体密度) は 2 粒子の絶対的な位置 $\mathbf{r}$, $\mathbf{r}'$ ではなく、2 粒子の相対的な位置 $\mathbf{r}' - \mathbf{r}$ に依存する。つまり、

$$g_{aa}^{(2)}(\mathbf{r}, \mathbf{r}') = g_{aa}^{(2)}(\mathbf{0}, \mathbf{r}' - \mathbf{r}), \quad (3.20)$$

のような関係が成り立つ。 $\mathbf{r}' - \mathbf{r}$ を改めて $\mathbf{r}$ とおき直しておき、空間分布関数 (spatial distribution function, SDF) を次式で定義する。

$$g_{aa}(\mathbf{r}) = g_{aa}^{(2)}(\mathbf{0}, \mathbf{r}). \quad (3.21)$$

これを角度方向に平均をとり、距離 r の分布関数に変換したものを動径分布関数 (radial distribution function, RDF) と呼ぶ。

$$g_{aa}(r) = \frac{1}{4\pi r^2} \int d\mathbf{r} \delta(|\mathbf{r}| - r) g_{aa}(\mathbf{r}). \quad (3.22)$$

もし、系が等方的である場合、

$$g_{aa}(|\mathbf{r}|) = g_{aa}(\mathbf{r}), \quad (3.23)$$

が成り立つ。SDF を計算することに意義が生じるのは、非等方系の場合や、一方の分子 (形状を持つ粒子) を特定の配向に固定した座標系で他方の粒子の位置を分布を調べるときである。これについては次節で述べる。

RDF は、一方の粒子から眺めて距離 r での他粒子の存在確率がバルクよりもどの程度高いかを表す。つまり、 $g_{aa}(r) > 0$ であれば、その距離で他粒子の存在確率はバルクよりも高く、 $g_{aa}(r) < 0$ の場合はバルクよりも存在確率が低いことを意味する。

3.6.1.2 異種粒子間の動径分布関数

粒子 a が N_a 個、粒子 b が N_b 個存在している系 (体積 V 、平均数密度はそれぞれの粒子で $\rho_a = N_a/V$, $\rho_b = N_b/V$) での、粒子 a と粒子 b の間の RDF を導入する。

$$\rho_{ab}^{(2)}(\mathbf{r}, \mathbf{r}') = \sum_{i=1}^{N_a} \sum_{j=1}^{N_b} \langle \delta(\mathbf{r} - \mathbf{r}_i^a) \delta(\mathbf{r}' - \mathbf{r}_j^b) \rangle. \quad (3.24)$$

ここで, $\mathbf{r}_i^p$ ($p = a, b$) は粒子 p の i 番目の粒子の位置である. 同種粒子間の場合と異なるのは, $j = i$ の場合を除外しなくても良い点である. これにより, 無相関の場合は,

$$\rho_{ab}^{(2)}(\mathbf{r}, \mathbf{r}') \approx \rho_a \rho_b, \quad (3.25)$$

となる. それ以外の点は全く同じなので, 同様の手続きを踏んで動径分布関数を導入できる. まず, 空間分布関数は,

$$g_{ab}(\mathbf{r}) = \frac{1}{\rho_a \rho_b} \rho_{ab}^{(2)}(\mathbf{0}, \mathbf{r}), \quad (3.26)$$

であり, これを角度方向について平均化することにより, 動径分布関数の表式を得る.

$$g_{ab}(r) = \frac{1}{4\pi r^2} \int d\mathbf{r} \delta(|\mathbf{r}| - r) g_{ab}(\mathbf{r}). \quad (3.27)$$

3.6.2 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-sel0	Common	-sel1	Common
-mode0	Common	-mode1	Common
-dr	Mode specific	-identical	Mode specific
-normalize	Mode specific	-separate_self	Mode specific

- **-dr <距離の刻み (Å)>**

- Default: 0.1

RDF を計算するときの距離の刻みを指定する.

- **-identical <true or false>**

- Default: false

-sel0 と -sel1 が同一の場合に true を指定する. 理論の解説で述べたように, RDF の計算は同種粒子間と異種粒子間で扱いが異なるので, ユーザー側でどちらの扱いにするかを指定する必要がある.

- **-normalize <true or false>**

- Default: true

RDF ($g_{12}(r)$) をそのまま出力するか (**true**), **-sel2** 側の部位の平均数密度 ρ_2 をかけたものを出力するか (**false**) を指定する. **true** の場合は, $\rho_{12}(r) = \rho_2 g_{12}(r)$ が出力される.

- **-separate_self** <true or false>

- Default: false

RDF を分子内と分子間の部分に分けるかどうかを指定する. このオプションが必要となるのは, 同種分子の異なる部位間の RDF を計算するときである. 例えば, 水分子 (原子ラベルを HW1-OW-HW2 とする) の OW 原子と HW1 原子の間の RDF を計算するときに, “-separate_self false” の条件で “-sel0 name OW”, “-sel1 name HW1`” と指定すると, RDF の中に同一分子内の OW-HW1 の寄与も含まれてしまうが, “-separate\_self true`” と指定することで, 同一分子内の寄与を分離することが可能である.

3.6.3 出力ファイルのフォーマット

- *.rdf ファイル

このファイルには, RDF が出力される. 出力内容は **-separate_self** によって異なる.

- “-separate_self false” の場合

- * 1 列目: 距離

- * 2 列目: RDF

- “-separate_self true” の場合

- * 1 列目: 距離

- * 2 列目: 異なる分子間の RDF

- * 3 列目: 同一分子内の RDF (自己相関関数と呼ばれる)

3.6.4 解析例

3.6.4.1 水分子の酸素原子間の RDF

```
#!/bin/bash
```

```
anatra rdf \
  -stype      parm7 \
  -sfile      complex.prmtop \
  -tintype    dcd \
  -tin        INP.dcd \
```

(次のページに続く)

(前のページからの続き)

```

-fhead      out          \
-sel0       resname WAT and name O    \
-sel1       resname WAT and name O    \
-mode0      residue        \
-mode1      residue        \
-dr         0.1            \
-identical  true           \
-normalize  true           \
-separate_self false

```

3.6.4.2 水分子の酸素原子-水素原子間の RDF

```
#!/bin/bash
```

```

anatra rdf          \
  -stype            parm7          \
  -sfile            complex.prmtop  \
  -tintype          dcd            \
  -tin              INP.dcd        \
  -fhead            out            \
  -sel0             resname WAT and name O    \
  -sel1             resname WAT and name H1    \
  -mode0            residue        \
  -mode1            residue        \
  -dr               0.1            \
  -identical        false          \
  -normalize        true           \
  -separate_self    true

```

3.6.5 タンパク質の C α 原子と水分子の酸素原子の間の RDF

タンパク質 (resid 1 to 192) の各 C α 原子と水分子の酸素原子の間の RDF を計算し、各 RDF を平均化したものが出力される。

```

anatra rdf          \
  -stype            parm7          \
  -sfile            complex.prmtop  \
  -tintype          dcd            \
  -tin              INP.dcd        \
  -fhead            out            \

```

(次のページに続く)

```

-sel0      resid 1 to 192 and name CA      \
-sel1      resname WAT and name O          \
-mode0     atom                            \
-mode1     residue                         \
-dr        0.1                             \
-identical false                          \
-normalize true                            \
-separate_self false

```

3.7 空間分布関数 (spatial_distr, sdf)

現バージョンでは gcc コンパイラを使った際に spatial_distr は使用できないので注意

3.7.1 理論背景

粒子 a が N_a 個存在している系 (体積 V , 数密度 $\rho_a = N_a/V$) を考える。ただし, 系の中心に分子が一つ配向も含めてピン止めされている状況を考える。この場合, ピン止めされた分子は外場の働きをする。このような状況は通常の溶液系のシミュレーションではありえないが, この節の最後で述べるようにトラジェクトリのフィッティングを行うことで, 仮想的に同様の状況を作り出すことができる。

動径分布関数の場合と同様に, 粒子 a の局所密度場を次式で定義する。

$$\hat{\rho}_a(\mathbf{r}) = \sum_{i=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i). \quad (3.28)$$

このアンサンブル平均は 1 体密度と呼ばれる。

$$\rho_a(\mathbf{r}) = \langle \hat{\rho}_a(\mathbf{r}) \rangle_{\text{ext}} \quad (3.29)$$

$$= \left\langle \sum_{i=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i) \right\rangle_{\text{ext}}. \quad (3.30)$$

ここで, $\langle \cdots \rangle_{\text{ext}}$ は外場存在下でのアンサンブル平均を表す。外場が存在しないとき $\rho_a(\mathbf{r}) = \rho_a$ となる。また, 中心の分子 (外場) から遠く離れているところでも, $\rho_a(\mathbf{r}) = \rho_a$ となる。空間分布関数 (spatial distribution function, SDF) を次式で定義する。

$$g_a(\mathbf{r}) = \frac{\rho_a(\mathbf{r})}{\rho_a}. \quad (3.31)$$

$g_a(\mathbf{r}) > 0$ はその位置での粒子 a の存在確率がバルクよりも高く, $g_a(\mathbf{r}) < 0$ は存在確率がバルクよりも低いことを意味する。

SDF を計算する際には, まず MD トラジェクトリ内に含まれている外場として扱う分子を参照構造に重なるように並進・回転させる (フィッティング)。また, アンラップされているトラジェクトリについては, あらかじめ PBC ラップを行っておく必要がある (図 3.2)。spatial_distr では, これら一連の操作を一括して行うことが可能で

ある。変換を行った後は、注目している分子は特定の位置・配向に固定された状態になるため、固定された外場として扱うことができ、周囲の分子の位置から SDF の計算が可能である。しかし、外場として扱う分子が、特定の安定構造を持たない場合は、フィッティングが適切に行えないため、SDF の計算は困難となる。天然構造を持つタンパク質については、シミュレーションの間に大きな構造変化が起きていなければ、多くの場合 SDF 計算は可能である。

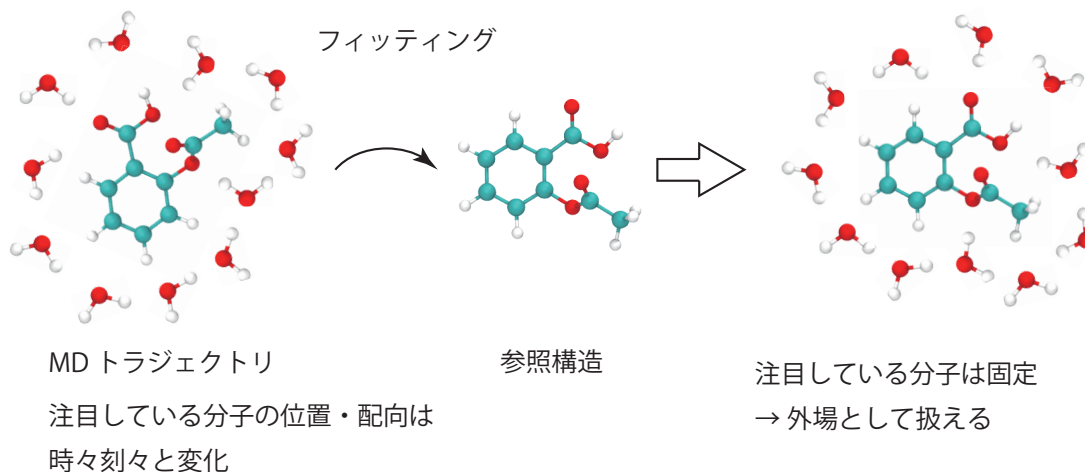


図 3.2: トラジェクトリ内の注目している分子を参照構造に重なるように系を並進・回転させることで (フィッティング), 特定の配向の分子周囲の他分子の 3 次元的な位置を調べることが可能である。spatial_distr では、これら一連の操作を一括して行うことが可能である。

3.7.2 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-selX	Common	-mode	Common
-ng3	Mode specific	-del	Mode specific
-origin	Mode specific	-fit	Mode specific
-refpdb	Mode specific	-refselid	Mode specific
-fitselid	Mode specific	-use_spline	Mode specific
-spline_resolution	Mode specific	-prep_only	Common

上記以外にも、条件付き空間分布など、より発展的な解析に対応したオプションも実装されているが、それらは次のアップデートの際に公式サポート予定である。

- **-ng3** < x, y, z 軸のグリッド数>

- Default: 50 50 50
- Example: -ng3 50 50 50

x, y, z 軸方向のグリッド数 n_x, n_y, n_z を指定する。従って、合計グリッド数は $n_x n_y n_z$ である。

- **-del** < x, y, z 軸のグリッド幅 (Å)>

- Default: 0.5 0.5 0.5
- Example: -del 0.5 0.5 0.5

x, y, z 軸のグリッド幅 $\Delta x, \Delta y, \Delta z$ (angstrom) を指定する。従って、グリッド点一つの体積要素 $\Delta V = \delta x \delta y \delta z$ である。

- **-origin** <原点の (x, y, z) 座標 (Å)>

- Default: 0.0 0.0 0.0
- Example: -origin 0.0 0.0 0.0

グリッドを張るときの原点を指定する。原点の座標を (x_0, y_0, z_0) とすると、グリッド点 i_x, i_y, i_z の座標 (x_i, y_i, z_i) は次式で与えられる。

$$x_i = x_0 + \Delta x (i_x - 1) \quad (3.32)$$

$$y_i = y_0 + \Delta y (i_y - 1) \quad (3.33)$$

$$z_i = z_0 + \Delta z (i_z - 1) \quad (3.34)$$

- **-fit** <true or false>

- Default: false
- Example: -fit true

外場として扱う分子 (溶質) 周囲を中心として、PBC ラップを行った後に、溶質分子が参照構造に重なるようにフィッティングする。-fit true の場合、次以降に示す-refpdb, -refselid, -fitselid を指定すると同時に、

-sel0 <SDF を計算したい分子の selection>

-sel1 <参照構造内のフィッティングに使う部位>

-sel2 <トラジェクトリ内のフィッティングに使う部位>

のように 3 つの selection を指定する必要がある。入力のトラジェクトリがフィッティング後の場合、必ず -fit false に設定しなければならない。フィッティング後のトラジェクトリに対して PBC ラップを行うと、不適切な分子配置になってしまうためである。

- **-refselid**

- Default: N/A
- Example: `-refselid 1`

照構造内のフィッティングに使う部位の `selid` を指定する。 `-fit` での例のように `-selX (x=0, 1, 2)` を指定した場合には、 `-refselid 1` とする。

• **-fitselid <フィッティングに使うトラジェクトリ内の部位に対応する selid>**

- Default: N/A
- Example: `-fitselid 2`

フィッティングに使うトラジェクトリ内の部位の `selid` を指定する。 `-fit` での例のように `-selX (x=0, 1, 2)` を指定した場合には、 `-fitselid 2` とする。

• **-use_spline <true or false>**

- Default: `false`
- Example: `-use_spline true`
- Note: スプラインによるグリッド細分化度は `-spline_resolution` で指定する。

SDF 計算の際、スプライン補間を行う。このプログラムでは Akima 補間を用いる。

• **-spline_resolution <スプラインによるグリッド細分化度>**

- Default: 4
- Example: `-spline_resolution 4`
- Note: "`-use_spline true`"のときに有効となる。

スプラインによるグリッド細分化の度合いを整数値で指定する。この値を n_r とし、スプライン実行前の SDF のグリッド刻みを $\Delta\alpha$ ($\alpha = x, y, z$) とすると、スプラインを施した SDF のグリッド刻みは

$$\Delta\alpha' = \Delta\alpha/n_r, \quad (3.35)$$

となる。また、スプライン実行前の合計グリッド数を N 、実行後のグリッド数を N' とすると、

$$N' = n_r^3 N, \quad (3.36)$$

となる。

3.7.3 出力ファイルのフォーマット

- *_out_g_original.dx

SDF が dx 形式で出力される。このファイルに格納されている SDF はスプラインが施されていない。

- *_out_g_spline.dx

SDF が dx 形式で出力される。このファイルに格納されている SDF はスプラインが施されている。"-use_spline true"のときのみこのファイルが出力される。

3.7.4 解析例

3.7.4.1 タンパク質周りの水分子の SDF

タンパク質 (resid 1 to 192) 周りの水分子 (重心) の SDF を計算する。

```
#!/bin/bash

anatra spatial_distr \
  -stype      parm7 \
  -sfile      complex.prmtop \
  -tintype     dcd \
  -tin        INP.dcd \
  -fhead      out \
  -sel0       water \
  -sel1       resid 1 to 192 and noh \
  -sel2       resid 1 to 192 and noh \
  -mode       residue \
  -ng3        50 50 50 \
  -del        0.4 0.4 0.4 \
  -origin     -10.0 -10.0 -10.0 \
  -fit        true \
  -refpdb     ref.pdb \
  -refselid   1 \
  -fitselid   2 \
  -use_spline true \
  -spline_resolution 4
```

3.8 平均二乗偏差 (rmsd)

3.8.1 理論背景

ここでは、平均二乗偏差（Root-mean-square deviation, RMSD）の定義について解説する。RMSD は注目する分子やその特定部位について、MD 計算中のある時刻での構造が、参照とする構造 (参照構造) からどの程度ずれているかを定量的に評価するものである。例えばあるタンパク質について、結晶構造を参照構造として、平衡化過程で参照構造からどの程度構造のズレが生じ、また構造が緩和しているかを判別する際によく用いられる。

原子 N_{sel} 個からなる分子について、 $\mathbf{r}_i$ を i 番目の原子の座標、 $\mathbf{r}^{\text{ref}}$ を参照構造中の原子の座標、 $\mathbf{r}(t)$ を時刻 t での座標とする。この時、平均二乗偏差 $\chi(t)$ は以下の式で定義される。

$$\chi(t) = \sqrt{\frac{1}{N_{\text{sel}}} \sum_{i=1}^{N_{\text{sel}}} |\mathbf{r}_i(t) - \mathbf{r}_i^{\text{ref}}|^2}. \quad (3.37)$$

3.8.2 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-fout	Common	-sel0	Common
-sel1	Common	-sel2	Common
-sel3	Common	-fit	Mode specific
-refpdb	Common	-fitselid	Mode specific
-refselid	Mode specific	-rmsdselid	Mode specific
-rmsdrefselid	Mode specific		

• -fout <出力ファイル>

- Default: out.rmsd
- Example: -fout out.rmsd

計算結果を出力するファイルを指定する。

• -refpdb <参照構造の PDB ファイル>

- Default: N/A
- Example: -refpdb ref.pdb

参照構造の PDB ファイルを指定する。この場合は RMSD 計算における参照構造である。

- **-fit <true or false>**

- Default: false
- Example: `-fit true`
- Note: true の場合、`-refselid`、`-fitselid` の指定が必要となる。

RMSD 計算の前にフィッティングを行うかどうかを指定する。

- **-fitselid <selid>**

- Default: N/A
- Example: `-fitselid 1`

- **-refselid <selid>**

- Default: N/A
- Example: `-refselid 2`

フィッティングを行う際、その対象部位に対応する `selid` を指定する。従って、このオプションは "`-fit true`" のときに必須となる。参照構造内のフィッティングに使う部位は "`-refselid`" で指定する。

- **-rmsdselid <selid>**

- Default: N/A
- Example: `-rmsdselid 2`

RMSD 計算を行う分子や部位に対応する `selid` を指定する。

- **-rmsdrefselid <selid>**

- Default: N/A
- Example: `-rmsdrefselid 3`

RMSD 計算を行う際の参照構造に対応する `selid` を指定する。

3.8.3 出力ファイルのフォーマット

- ***.rmsd** ファイル

このファイルには、RMSD が出力される。

- 1 列目: 時間
- 2 列目: RMSD

3.8.4 解析例

3.8.4.1 水中におけるタンパク質の結晶構造からの RMSD

タンパク質 (resid 1 to 161) の $C\alpha$ 原子に対する RMSD を計算する.

```
anatra rmsd \
  -stype      parm7 \
  -sfile      complex.prmtop \
  -tintype    dcd \
  -tin        INP.dcd \
  -fout       out.rmsd \
  -sel0       resid 1 to 161 and name CA \
  -sel1       resid 1 to 161 and name CA \
  -sel2       resid 1 to 161 and name CA \
  -sel3       resid 1 to 161 and name CA \
  -fit        true \
  -refpdb     ref.pdb \
  -fitselid   0 \
  -refselid   1 \
  -rmsdselid  2 \
  -rmsdrefselid 3
```

3.9 相互作用エネルギー (interaction_energy, ene)

`interaction_energy (ene)` では選択した分子やその集団, もしくは特定部位等の間に働く相互作用エネルギーを計算できる.

現在 **anatra** では以下の三種類の相互作用が計算可能である.

- 静電相互作用

分子 A と分子 B の間に働く静電相互作用 U_{elec} について, 2 通りのスキームを実装している. 一つは Bare 相互作用であり, i と j はそれぞれ分子 A と B の原子インデックス番号とすると A-B 間の相互作用は

$$U_{\text{elec}} = \sum_{i \in A} \sum_{j \in B} \frac{q_i q_j}{r_{ij}}, \quad (3.38)$$

で表される. もう一つは, Smooth particle mesh Ewald (SPME) 法による静電相互作用の計算である. 実際の MD 計算は長距離的な静電相互作用を考慮するために SPME 法を用いるのが現在の標準であり, MD 計算と正しく対応する静電相互作用の計算が必要な場合は SPME 法を用いた計算が必要となる.

- Lennard-Jones (LJ) 相互作用

van der Waals 相互作用を表現するための経験的なモデルである Lennard-Jones 相互作用は, 以下の式で定義される.

$$U_{\text{LJ}} = \sum_{i \in A} \sum_{j \in B} 4\epsilon_{ij} \left\{ \left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right\}. \quad (3.39)$$

この式中の ϵ_{ij} と σ_{ij} は分子 A 中の原子 i と分子 B 中の原子 j との間の LJ パラメータである。

- **Attractive LJ 相互作用** Attractive LJ 相互作用は、以下で定義される。

$$U_{\text{attr}} = \sum_{i \in A} \sum_{j \in B} u_{\text{attr},ij}, \quad (3.40)$$

$$u_{\text{attr},ij} = \begin{cases} 4\epsilon_{ij} \left\{ \left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right\} & r_{ij} \geq 2^{1/6}\sigma \\ -\epsilon_{ij} & r_{ij} < 2^{1/6}\sigma. \end{cases} \quad (3.41)$$

これは LJ 相互作用の極小部分を引き伸ばした形の関数であり (図 3.3 参照), LJ 相互作用の関数で生じていた, エネルギーに対する距離の多価性を取り除いたものである。分子会合過程等の化学過程について PMF 作成を行う際, Attractive LJ 相互作用を反応座標として用いれば, LJ 相互作用に比べて明確に解離や会合などの状態区別が可能となる。

以上の 3 種類の相互作用計算に加え, 静電相互作用と LJ 相互作用, もしくは静電相互作用と attractive LJ 相互作用の和を計算することも可能である。

3.9.1 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-sel0	Common	-sel1	Common
-mode0	Common	-mode1	Common
-parmformat	Mode specific	-fanaparm	Mode specific
-pbc	Mode specific	-calc_vdw	Mode specific
-calc_elec	Mode specific	-vdw_type	Mode specific
-elec_type	Mode specific	-dt	Mode specific
-rvdwcut	Mode specific	-relcut	Mode specific
-pme_alpha	Mode specific	-pme_grids	Mode specific
-pme_rigid	Mode specific	-pme_dual	Mode specific
-pme_fprm	Mode specific	-prep_only	Common

- **-parmformat <prmtop or anaparm>**

- Default: prmtop
- Example: -parmformat anaparm
- Note : anaparm を指定した場合, -fanaparm の指定が必要となる。

電荷情報を参照するパラメータファイルを指定する。 `prmtop` を指定する場合は `-sfile` にて指定した `prmtop` ファイルを自動で参照する。

- **-fanaparm <anaparm ファイル名>**

- Default: N/A
- Example: `-fanaparm complex.anaparm`

LJ パラメータや電荷情報を参照する `anaparm` ファイル名を指定する。このオプションは `-parmformat anaparm` のときに必要となる。

- **-pbc <true or false>**

- Default: `false`
- Example: `-pbc true`

周期境界を考慮して相互作用計算を行うかを指定する。つまり `box` の上下左右に同様の `box` が存在するとしてその中のミラー分子からの寄与も計算するかを決定する。

- **-calc_vdw <true or false>**

- Default: `true`
- Example: `-calc_vdw false`

van der Waals (LJ) 相互作用を計算するかを指定する。

- **-calc_ele <true or false>**

- Default: `false`
- Example: `-calc_ele true`

静電相互作用を計算するかどうかを指定する。

- **-vdw_type <standard or attractive>**

- Default: `standard`
- Example: `-vdw attractive`

計算する van der Waals 相互作用の関数を選択する。 `standard` は通常の LJ 相互作用 (式 (3.39)) を計算し、 `attractive` は U_{attr} (式 (3.40)) を計算する

- **-elec_type <bare or pme>**

- Default: `bare`

- Example: `-elec bare`

静電相互作用の計算方法を指定する。bare の場合には、式 (3.38) に基づいて計算が行われる。pme を指定した場合、Smooth particle mesh Ewald (PME) 法に基づいて計算が行われる。

- **-dt <時間刻み>**

- Default: 0.1
- Example: `-dt 1.0`

時系列データ出力の際の時間軸（第 1 列）の時間刻みを指定する。

- **-rvdwcut <カットオフ距離 (Å)>**

- Default: 12.0
- Example: `-rvdwcut 20.0`

vdW 相互作用計算の際のカットオフ距離を指定する。

- **-relcut <カットオフ距離 (Å)>**

- Default: 1.0e10
- Example: `-relcut 1.0e10`

静電相互作用計算の際のカットオフ距離を指定する。

- **-pme_alpha <スクリーニングパラメータ (Å⁻¹)>**

- Default: 0.35e0
- Example: `-pme_alpha 0.35e0`

PME 法におけるスクリーニングパラメータの値を指定する。-elec pme の時のみ参照される値である。

- **-pme_grids <x, y, z 軸のグリッド数>**

- Default: 64 64 64
- Example: `-pme_grids 64 64 64`

x, y, z 軸方向のグリッド数 n_x, n_y, n_z を指定する。従って、合計グリッド数は $n_x n_y n_z$ である。

- **-pme_rigid <true or false>**

- Default: false

- Example: `-pme_rigid false`

SPME 計算を行う際、`-sel0` で指定した部位が空間上に固定されている場合に、`-pme_rigid true` と設定することで、計算負荷を削減することができる。

3.9.2 出力ファイルのフォーマット

- *.uij ファイル

選択部位間の静電相互作用が出力される。 `-mode 0`, `-mode1` の指定により、部位 0,1 がそれぞれ n_1 個, n_2 個の分子に分割されたとすると、合計で $n_1 n_2$ 通りの相互作用が出力されることとなる。この時、各データは以下のように出力される。

- 1 列目: 時間軸
- 2 列目: 分子対 (1, 1) の相互作用
- 3 列目: 分子対 (1, 2) の相互作用…
- n_2 列目: 分子対 (1, n_2) の相互作用
- $n_2 + 1$ 列目: 分子対 (2, 1) の相互作用
- $n_2 + 2$ 列目: 分子対 (2, 2) の相互作用…

3.9.3 解析例

ここではよく行う解析について最低限のオプションを使ったスクリプトの例を示しておく。

3.9.3.1 溶質間の静電相互作用と LJ 相互作用の和

タンパク質 (resid 1 to 161) とリガンド分子 (resname LIG) の間の静電相互作用+LJ 相互作用の和を計算する場合を示す。ここでは、静電相互作用の計算に SPME 法を用いる。

```
#!/bin/bash

anatra energy
  -stype      parm7                \
  -sfile      complex.prmtop       \
  -tintype    dcd                  \
  -tin        INP.dcd              \
  -fhead      out                   \
  -parmformat prmtop               \
  -pbc        true                  \
  -calc_vdw   true                  \
```

(次のページに続く)

(前のページからの続き)

```

-calc_elec    true          \
-vdw_type     standard      \
-elec_type    pme           \
-mode0        whole         \
-mode1        residue       \
-dt           0.1           \
-rvdwcut      12.0          \
-relcut       12.0          \
-pme_alpha    0.35          \
-pme_grids    64 64 64      \
-pme_rigid    false         \
-sel0         resid 1 to 161 \
-sel1         resname LIG    \
-prep_only    false         \

```

3.9.3.2 溶質間の LJ 相互作用

タンパク質 (resid 1 to 161) とリガンド分子 (resname LIG) の間の LJ 相互作用を計算する場合を示す.

```

#!/bin/bash

anatra energy
  -stype      parm7          \
  -sfile      complex.prmtop \
  -tintype    dcd            \
  -tin        INP.dcd        \
  -fhead      out            \
  -parmformat prmtop         \
  -pbc        true           \
  -calc_vdw   true           \
  -calc_elec   false         \
  -vdw_type    standard      \
  -mode0       whole         \
  -mode1       residue       \
  -dt          0.1           \
  -rvdwcute    12.0          \
  -relcut      1.0e10        \
  -sel0        resid 1 to 161 \
  -sel1        resname LIG    \
  -prep_only   false         \

```

3.10 脂質膜秩序 (lipid_order, scd)

lipid_order (scd) ではリン脂質のアシル鎖に関する配向秩序パラメータ S_{CD} の計算が可能である。脂質膜の法線方向を z として、特定の炭素原子と隣接する水素原子のなす角を θ と定義する。 S_{CD} は θ を用いて式で定義される。

$$S_{CD} = \frac{1}{2} \langle 3 \cos^2 \theta - 1 \rangle. \quad (3.42)$$

S_{CD} の値域は $0 \leq S_{CD} \leq 1$ であり、その値が大きいほど秩序性が高く、0 は無秩序であることを意味する。 S_{CD} は炭素ごとに定義されるので、炭素原子の位置に応じた膜秩序の解析が可能である。

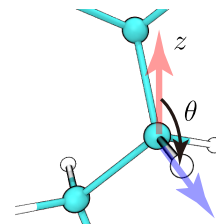


図 3.3: z 軸と特定の炭素原子と水素原子を結ぶベクトルのなす角を θ と定義する。 θ は脂質の局所的な配向を表す。

3.10.1 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-selX	Common	-mode	Common
-cindex	Mode specific	-prep_only	Common

• -selX <部位選択>

- Default: N/A
- Sample: -sel0 name C32 H2X H2Y

炭素原子および隣接する水素原子を指定する。 S_{CD} を計算したい炭素原子ごとに -selX で指定する。

• -cindex < Carbon index >

- Default: N/A
- Sample: -cindex 1 2 3 4 5

-sel0, -sel1, ... で指定した炭素原子が、アシル鎖の付け根から数えて何番目か (Carbon index) をこのオプションで指定する。したがって、-selX で定義した selid の数分だけ番号を並べる必要がある。このオプションで指定した Carbon index は、出力ファイル*.scd の x 軸 (1 列目) として用いられる。

3.10.2 出力ファイルのフォーマット

- *.scd

S_{CD} を -sel0, -sel1, ... の順番で指定した炭素原子の順番で出力する.

- 1 列目: Carbon index (-cindex で指定)
- 2 列目: S_{CD}

3.10.3 解析例

3.10.3.1 DPPC 脂質膜の S_{CD} 計算

```
anatra lipid_order \
  -stype      psf \
  -sfile      complex.psf \
  -tintype    netcdf \
  -flist_traj flist \
  -fhead      run_nc \
  -dt         0.1 \
  -cindex     1 2 3 4 5 \
  -sel0       name C23 H3R H3S and segid MEMB \
  -sel1       name C24 H4R H4S and segid MEMB \
  -sel2       name C25 H5X H5Y and segid MEMB \
  -sel3       name C26 H6X H6Y and segid MEMB \
  -sel4       name C27 H7X H7Y and segid MEMB \
  -prep_only  false
```

3.11 界面法線方向 (z) に沿った分布関数 (z_profile, zprof)

z_profile (zprof) は, 膜界面の法線方向 (z) に沿った分布を計算する. 分子 a の z 軸に沿った局所密度場を $\hat{\rho}_a(z)$ と定義する.

$$\hat{\rho}_a(z) = \frac{1}{A_{xy}} \sum_{i=1}^{N_a} \delta(z - z_i). \quad (3.43)$$

ここで, A_{xy} はシミュレーションセルの xy 面の面積, z_i は i 番目の分子の z 座標である. これの平均をとったものが, 分布関数である.

$$\rho_a(z) = \langle \hat{\rho}(z) \rangle. \quad (3.44)$$

実験データと比較するために, 電子密度分布 $\rho_a^e(z)$ を計算することも多い. その場合の局所密度場は

$$\hat{\rho}_a^e(z) = \sum_{i=1}^{N_a} \sum_{\lambda=1}^{N_s} q_{\lambda} \delta(z - z_{i,\lambda}), \quad (3.45)$$

で与えられる。ただし、 q_λ は分子 a 内の λ 番目の原子が持つ電子数、 $z_{i,\lambda}$ は i 番目の分子 a 内にある λ 番目の原子の z 座標である。電子密度分布は $\rho_a^e(z) = \langle \rho_a^e(z) \rangle$ と定義される。

3.11.1 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-sel0	Common	-sel1	Common
-mode0	Common	-mode1	Common
-target_selid	Mode specific	-center_selid	Mode specific
-denstype	Mode specific	-symmetrize	Mode specific
-prep_only	Common	—	—

- **-selid0 <selid>**

- **-selid1 <selid>**

- Default: N/A
- Example: `-selid0 resname ETOH -selid1 segid MEMB`

分布関数を計算したい分子を選択する。また、 $z = 0$ を膜中心に設定したい場合は、膜分子も追加で選択する。

- **-target_selid <selid>**

- Default: N/A
- Example: `-target_selid 0`

-sel x のうち、分布関数を計算したい分子に対応する selid を指定する。

- **-center_selid <selid>**

- Default: false
- Example: `-target_selid 1`

-sel x のうち、膜分子に対応する selid を指定する。この selid で指定された膜分子の重心が $z = 0$ に設定される。

- **-symmetrize <true or false>**

- Default: false
- Example: `-symmetrize false`

$z < 0$ と $z > 0$ の分布を平均して対称化するかどうかを指定する.

- **-denstype <number or electron>**

- Default: number
- Example: `-denstype number`

数密度分布 (number) か電子密度分布 (electron) のどちらを計算するかを指定する.

3.11.2 出力ファイルのフォーマット

- ***.zprof**

`-target_selid` で指定した `selid` の分子の分布関数が出力される.

- 1 列目: z 座標
- 2 列目: 分布関数

3.11.3 解析例

3.11.3.1 脂質膜-水系における水分子の数密度分布

```
#!/usr/bin/env bash
```

```
anatra z_profile \
  -stype      psf \
  -sfile      complex.psf \
  -tintype    netcdf \
  -flist_traj flist \
  -fhead      run \
  -sel0       water \
  -sel1       segid MEMB and noh \
  -mode0      residue \
  -mode1      whole \
  -target_selid 0 \
  -center_selid 1 \
  -denstype   number \
  -prep_only  false
```

3.12 特定の方向 z に対する分子の配向 (z_orient, zori)

z_orient (zori) は、特定の方向 (z) を基準として、分子の配向分布を計算する。分子内にある 2 つの部位 **bottom**, **head** を定義し、それぞれの重心を $\mathbf{R}_B, \mathbf{R}_H$ とおく。そして、分子軸 $\mathbf{u}$ を

$$\mathbf{u} = \mathbf{R}_H - \mathbf{R}_B, \quad (3.46)$$

と定義する。 z 軸 (単位ベクトル $\mathbf{e}_z$) と $\mathbf{u}$ (単位ベクトル $\mathbf{e}_u$) のなす角を θ とすると、

$$\cos \theta = \mathbf{e}_z \cdot \mathbf{e}_u, \quad (3.47)$$

である。 $\cos \theta$ に対する分布関数を

$$P(\chi) = C \langle \delta(\chi - \cos \theta) \rangle, \quad (3.48)$$

とする。ただし、 C は規格化定数であり、

$$\int_0^\pi d\theta \sin \theta P(\chi) = 1, \quad (3.49)$$

を満たすように設定する。上式は、 θ から $\chi = \cos \theta$ への変数変換を考えることで、次式のように書き直せる。

$$\int_0^1 d\chi P(\chi) = 1. \quad (3.50)$$

z_orient (zori) は、上式を満たす $P(\chi) = P(\cos \theta)$ を計算する。

さらに、実験室座標系での z 軸に加えて、特定の分子の向きによって定義されるベクトルを z 方向に設定することも可能である。その分子の 4 つの部位の重心を $\mathbf{x}_{B_0}, \mathbf{x}_{H_0}, \mathbf{x}_{B_1}, \mathbf{x}_{H_1}$ として、ベクトル

$$\mathbf{v}_0 = \mathbf{x}_{H_0} - \mathbf{x}_{B_0} \quad (3.51)$$

$$\mathbf{v}_1 = \mathbf{x}_{H_1} - \mathbf{x}_{B_1} \quad (3.52)$$

を用意することで、 z 方向の単位ベクトル

$$\mathbf{e}_z = \frac{\mathbf{v}_1 \times \mathbf{v}_2}{|\mathbf{v}_1 \times \mathbf{v}_2|} \quad (3.53)$$

を定義する。このようにして定義した $\mathbf{e}_z$ と注目している分子の分子軸ベクトル $\mathbf{u}$ のなす角によって配向を定義することで、配向分布 $P(\chi)$ の定義・計算が可能である。

3.12.1 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-sel x	Common	-mode x	Common
-zdef_type	Mode specific	-vecX_bottom_selid	Mode specific
-vecX_head_selid	Mode specific	-bottom_selid	Mode specific
-head_selid	Mode specific	-judgeup	Mode specific
-nmolup	Mode specific	-center_selid	Mode specific
-dx	Mode specific	-dt	Mode specific
-prep_only	Common	-	-

- **-sel x <selelection>** (最大 7 個)

- Default: N/A
- Example: -sel0 name P -sel1 name N

配向を定義するための部位 bottom, head を指定する。実験室座標系ではなく、特定の分子の配向によって決まる方向を z 方向に用いるときは (-zdef_type を参照), ベクトル $\mathbf{v}_1, \mathbf{v}_2$ (式 (3.51)), (3.52)) を定義するための部位選択を行う必要がある。また、実験室座標系の z 軸を用いる場合において、 $\mathbf{e}_z$ の向きを特定分子の重心の z 座標を基準にして反転させるときには (-judgeup を参照), その分子の選択も必要となる。

- **-head_selid <selid>**

- **-bottom_selid <selid>**

- Default: N/A
- Example: -head_selid 0 -bottom_selid 1

分子軸 $\mathbf{u}$ (式 (3.46)) を定義するための部位 bottom, head に対応する部位をそれぞれ -bottom_selid, -head_selid で指定する。

- **-zdef_type <coord or outerprod>**

- Default: N/A
- Example: -zdef_type coord

- **-vec x _bottom_selid <selid>** ($x = 0, 1$)

- **-vecx_head_selid <selid>** ($x = 0, 1$)

z 方向の定義を指定する. 実験室座標系の z 軸を z 方向に設定する場合は `coord` を, 特定の分子の配向によって z 方向を定義する場合は `outerprod` を指定する. `outerprod` の場合は, `-vecx_bottom_selid`, `-vecx_head_selid` を設定する必要がある. これらのオプションは, 式 (3.51), (3.52) の

- x_{B_0} : `-vec0_bottom_selid`
- x_{H_0} : `-vec0_head_selid`
- x_{B_1} : `-vec1_bottom_selid`
- x_{H_1} : `-vec1_head_selid`

に対応している.

- **-judgeup <nmolup coord or none>**

- Default: `coord`
- Example: `-judgeup coord`

- **-nmolup <# of target molecules in upper leaflet>**

- Default: 0
- Example: `-nmolup 100`

- **-center_selid <selid>**

- Default: N/A
- Example: `-center_selid 2`

実験室座標系の z 軸を基準にとったときの, $\mathbf{e}_z$ の向きをこのオプションで指定する. z 軸の正の方向を向いた単位ベクトルを $\mathbf{e}'_z$ とする.

`-judgeup nmolup` の場合: オプション `-nmolup` で指定した数を N_{up} とすると, 最初の N_{mol} 分子は $\mathbf{e}_z = \mathbf{e}'_z$ として $\cos \theta$ を計算し, それ以外の分子は $\mathbf{e}_z = -\mathbf{e}'_z$ として $\cos \theta$ を計算する.

`-judgeup coord` の場合: `-center_selid` で指定した分子の重心の z 座標を z_c , `-bottom_selid` と `-head_selid` で指定された部位の重心の z 座標をそれぞれ z_B, z_H とすると,

$$\mathbf{e}_z = \begin{cases} \mathbf{e}'_z & \frac{z_H + z_B}{2} > 0 \\ -\mathbf{e}'_z & \frac{z_H + z_B}{2} \leq 0 \end{cases} \quad (3.54)$$

のように, 注目している分子の位置に応じて, $\mathbf{e}_z$ の向きを設定する.

- **-dx <グリッド幅 (Å)>**

- Default: 0.1

- Example: `-dx 0.1`

$P(\chi)$ を計算するときのグリッド幅 ($\Delta\chi$) を指定する.

- **-dt <時間刻み>**

- Default: 1
- Example: `-dt 1`

時系列データ出力の際の時間軸 (第 1 列) の時間刻みを指定する.

3.12.2 出力ファイルのフォーマット

- ***.costheta**

$\cos \theta$ の時系列データが出力される.

- 1 列目: 時間
- 2 列目: 1 番目の分子の $\cos \theta$
- 3 列目: 2 番目の分子の $\cos \theta$...

- ***.distr**

$P(\chi) = P(\cos \theta)$ が出力される.

- 1 列目: x 軸 ($\cos \theta$)
- 2 列目: $P(\cos \theta)$

3.12.3 解析例

3.12.3.1 脂質二分子膜内における脂質分子の配向分布

```
#!/usr/bin/env bash

anatra z_orient \
  -stype      psf \
  -sfile      complex.psf \
  -tintype    nc \
  -flist_traj flist \
  -fhead      run_nc \
  -judgeup    coord \
  -dx         0.1 \
```

(次のページに続く)

(前のページからの続き)

```

-dt          0.1          \
-sel0        name P        \
-sel1        name N        \
-sel2        segid MEMB and noh \
-mode0       residue       \
-mode1       residue       \
-mode2       whole         \
-bottom_selid 0            \
-head_selid  1            \
-center_selid 2           \
-prep_only   false

```

3.13 クラスターサイズ (cluster_size)

`cluster_size(cls)` は、系内に含まれるクラスターのサイズ分布を計算する。分子 i, j の位置をそれぞれ $\mathbf{r}_i, \mathbf{r}_j$, そしてカットオフ距離を r_{cut} として,

$$|\mathbf{r}_i - \mathbf{r}_j| \leq r_{\text{cut}} \quad (3.55)$$

を満たすとき、分子 i と j は同じクラスターに属していると定義する。クラスターに属する分子の数 n がクラスターのサイズである。クラスターのサイズ分布関数 $P(n)$ を、次式に示す規格化条件を満たすものとして定義する。

$$\sum_{n=1}^{N_{\text{tot}}} P(n) = 1 \quad (3.56)$$

ここで、 N_{tot} は系内に含まれる注目分子の数である。各分子がどのクラスターに属するかは、union-find tree によって決定される。

3.13.1 オプション一覧

オプション	備考	オプション	備考
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-sel0	Common	--mode	Common
-rcut	Mode specific	-	-

- -rcut <カットオフ距離 (Å)>

- Default: 3.5
- Example: `-dr 3.5`

2つの分子が同じクラスターに属しているかどうかを規定するカットオフ距離 r_{cut} を指定する. 式 (3.55) に示す通り, 分子 $i-j$ 間距離が r_{cut} 以下のとき, 分子 i, j が同じクラスターに属していると判定される.

3.13.2 出力ファイルのフォーマット

- `*.distr`

クラスターサイズ n に対する分布 $P(n)$ が出力される.

- 1 列目: クラスターサイズ n
- 2 列目: $P(n)$
- 3 列目: $P(n)$ の running integral

3.13.3 解析例

3.13.3.1 水クラスターのサイズ分布

```
#!/usr/bin/env bash

anatra cluster_size \
  -stype      psf      \
  -sfile      complex.psf \
  -tintype    nc       \
  -flist_traj flist     \
  -fhead      run_nc   \
  -sel0       water and name 0 \
  -mode0      residue  \
  -prep_only  false
```

第4章 fanatra の使い方

4.1 はじめに

fanatra は"Fortran ANATRA"の略であり、その名の通り Fortran を用いて解析を行うプログラムである。この章では **fanatra** の使い方を解説する。**fanatra** は VMD を通さずに解析を行うプログラム群であるため、計算ノード上での VMD の利用が難しい計算機システムでも利用が可能となる。**fanatra -h** を実行することでその種類の一覧が表示される。

```
$ fanatra -h
Available analysis in Fortran ANATRA
histogram          (hist)   : Histogram Analysis
potential_of_mean_force (pmf)   : Free-energy Profile Analysis
restricted_radial    (rrpmf)  : Restricted Radial Analysis
extract_frame       (extf)   : Extract Frames Using CV
random_frame        (randf)  : Randomly Extract Frames
state_define        (sdef)   : Define States From Time-series Data
moving_average      (movave)  : Moving Average Analysis
average_function    (avef)   : Function Average Analysis
bootstrap_prepper   (boot)   : Bootstrap Sample Generator
return_prob         (rp)     : Returning Probability Analysis
rprate              (rprate)  : Rate Constant Analysis based on RP theory
iepdyn              (iepdyn) : Integral-equation Formalism of Population Dynamics
```

上記のうち、**histogram** などが解析モードであり、省略して **hist** として入力することも可能である。**anatra** がコマンドライン上でパラメータを指定して解析可能だったのに対して、**fanatra** ではパラメータファイルを別に用意して読み込ませる形で実行する。例えば、

```
$ fanatra hist コントロールファイル
```

のように実行する。読み込ませるパラメータファイルは Fortran の namelist 形式である。また、各解析モードでのパラメータファイルの形式やその内容は "**fanatra <解析モード> -h**" で簡単な説明とサンプルが表示されるため大体の使い方はそれで理解ができる。**histogram** を例を以下に示す。

```
$ fanatra histogram -h

=====
```

(次のページに続く)

```
Histogram Analysis

=====
&input_param
  fcv      = "cvdata1" "cvdata2" ! Time-series CV data
  flist_cv = "list" ! CV-file list (Either fcv or flist_cv should be specified)
/
&output_param
  fhead = "run" ! header of output filename
/
&option_param
  dx      = 0.1d0 ! grid spacing
  xsta    = 0.0d0 ! minimum value of the coordinate
  nx      = 100   ! number of grids
  ncol    = 1     ! number of data column
/

::: format of time-series data :::
1st column: time (arbitrary), Nth column (N>1): data
example)

0.00    0.02120
0.01    0.03140
0.02    0.22310
....    .....
```

4.2 共通パラメータ

fanatra におけるパラメータファイルのほとんどは Fortran の namelist 形式に依拠しており, **input_param** セクション, **output_param** セクション, **option_param** セクションに分けてパラメータを指定する. 全ての解析モードで共通している **input_param** と **output_param** について, ここで示しておく.

4.2.1 &input_param

この namelist では、入力ファイルに関するパラメータを指定する。

パラメータ名	機能	例
ftraj	入力トラジェクトリ名を指定 (複数指定可)	ftraj = "inp1.dcd", "inp2.dcd"
fcv	入力時系列データ名を指定 (複数指定可)	fcv = "cv1.txt", "cv2.txt"
fweight	入力時系列データに対応する各ステップでの統計重みを指定 (複数指定可)	fweight = "weight1.txt", "weight2.txt"
flist_traj ¹	入力トラジェクトリの一覧を格納したファイルを指定	flist_traj = "list.txt"
flist_cv ²	入力時系列データの一覧を格納したファイルを指定	flist_cv = "list.txt"
flist_weight ³	統計重みファイルの一覧を格納したファイルを指定	flist_weight = "list.txt"

¹ ftraj と flist_traj の両方を指定した場合、flist_traj の内容が優先される。

² fcv と flist_cv の両方を指定した場合、flist_cv の内容が優先される。

³ fweight と flist_weight の両方を指定した場合、flist_weight の内容が優先される。

• ftraj

- Default: N/A
- Example: ftraj = "inp1.dcd", "inp2.dcd"

入力のトラジェクトリファイルを指定する。例に示す通り、複数のファイルを指定することも可能である。ファイル数が膨大な場合は、flist_traj で指定することも可能である。

• fcv

- Default: N/A
- Example: fcv = "cv1.txt", "cv2.txt"

入力の時系列データファイルを指定する。例に示す通り、複数のファイルを指定することも可能である。ファイル数が膨大な場合は、flist_cv で指定することも可能である。

• fweight

- Default: N/A
- Example: fweight = "cv1.txt", "cv2.txt"

入力の統計重みファイルを指定する。統計重みファイルでは、時系列データの各ステップでの統計的重みを与えることができる。統計的重みとしてエンハンスドサンプリング法や Multistate Bennett Acceptance Ratio (MBAR) 法によって得られたものを入力することが可能である。統計重みの総和値はプログラム内で自動的

に 1 に規格化される。例に示す通り、複数のファイルを指定することも可能である。ファイル数が膨大な場合は、`flist_weight` で指定することも可能である。

- **flist_traj**

- Default: N/A
- Example: `flist_traj = "list.txt"`

入力のトラジェクトファイルのリストを格納したファイルを指定する。`flist_traj` で指定された内容が、`ftraj` で指定した内容よりも優先される。リストファイルには、下記の例の通り、入力のトラジェクトリファイルが 1 行ごとに記されている必要がある。

```
inp1.dcd
inp2.dcd
inp3.dcd
inp4.dcd
```

- **flist_cv**

- Default: N/A
- Example: `flist_cv = "list.txt"`

入力の時系列データファイルのリストを格納したファイルを指定する。`flist_cv` で指定された内容が、`fcv` で指定した内容よりも優先される。リストファイルには、下記の例の通り、入力の時系列データファイルが 1 行ごとに記されている必要がある。

```
inp1.txt
inp2.txt
inp3.txt
inp4.txt
```

- **flist_weight**

- Default: N/A
- Example: `flist_weight = "list.txt"`

入力の統計重みデータファイルのリストを格納したファイルを指定する。`flist_weight` で指定された内容が、`fweight` で指定した内容よりも優先される。リストファイルには、下記の例の通り、入力の統計重みファイルが 1 行ごとに記されている必要がある。

```
inp1.txt
inp2.txt
inp3.txt
inp4.txt
```

4.2.2 時系列データのフォーマット

時系列データのフォーマットは下記である。

- 1 列目: 時間軸
- 2 列目: 1 つめの時系列データ
- 3 列目: 2 つめの時系列データ

1 列目は、全ての解析モードにおいて読み込まれず、2 列目から 1 つめのデータ列としてカウントされる。

4.2.3 統計重みデータのフォーマット

統計重みデータのフォーマットは下記である。

- 1 列目: 時間軸
- 2 列目: 統計重み値

1 列目は、全ての解析モードにおいて読み込まれず、2 列目のみ利用される。統計重みデータの行数は対応する時系列データの行数と一致している必要がある。

4.2.4 &output_param

この namelist では、出力に関するパラメータを指定する。

パラメータ名	機能	例
fhead	出力ファイル名の拡張子以外の部分を指定	fhead = "run"
file_extension ¹	出力ファイル名の拡張子を指定	file_extension = "dat"

¹ このパラメータが有効化される解析モードは多くない。

• fhead

- Default: run
- Example: fhead = "run"

出力するファイル名の拡張子を除いた部分 (basename) を指定する。例えば、fhead = run と指定した場合、次に述べる解析モード histogram は、デフォルトでは run.dat というファイルを出力する。

• file_extension

- Default: dat

- Example: `file_extension = "dat"`

出力するファイル名の拡張子を指定する。ただし、このパラメータが有効化される解析モードは、`histogram` などごく一部である。

4.3 ヒストグラム (histogram, hist)

この解析モードでは、何らかの時系列データ $\xi(t)$ から分布関数 $P(x)$ を計算する。

$$P(x) = \frac{1}{N} \langle \delta(x - \xi) \rangle \quad (4.1)$$

ここで、 N はデータ点の数であり、上式は規格化条件

$$\int_{-\infty}^{\infty} dx P(x) = 1 \quad (4.2)$$

を満たす。

4.3.1 パラメーター一覧

Namelist	パラメータ	備考
&input_param	<code>flist_cv</code>	—
	<code>fcv</code>	—
&output_param	<code>fhead</code>	—
	<code>file_extension</code>	—
&option_param	<code>dx</code>	分布関数のグリッド幅
	<code>xsta</code>	グリッドを張るスタート位置
	<code>nx</code>	グリッド数
	<code>ncol</code>	1 ファイルあたりのデータ列数

4.3.2 &option_param

- **dx**

- Default: 0.1
- Example: `dx = 0.1`

分布関数を計算する際のグリッド幅 Δx を指定する。

- **xsta**

- Default: 0.0

- Example: `xsta = 0.0`

グリッドを張るスタート位置 x_0 を指定する. i 番目のグリッドの位置を x_i とすると,

$$x_i = x_0 + (i - 1) \Delta x \quad (4.3)$$

である.

- **ncol**

- Default: 1
- Example: `ncol = 10`

1 つの時系列データファイルあたりに含まれるデータ列数を指定する.

4.3.3 出力ファイルのフォーマット

- `*.dat` (拡張子は `file_extension` によって変更可)
 - 1 列目: x 座標
 - 2 列目: 分布関数 $P(x)$

4.3.4 解析例

3 つのデータ列を持つ時系列データ `inp.ts` から, $5 \leq x \leq 10$ の範囲で分布関数 $P(x)$ を計算する例を以下に示す.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
fcv = "inp.ts"
/
&output_param
fhead = "run"
/
&option_param
dx    = 0.1d0
xsta  = 5.0d0
nx    = 50
ncol  = 3
/
EOF
```

(次のページに続く)

```
fanatra hist run.inp
```

4.4 平均力ポテンシャル (potential_of_mean_force, pmf)

この解析モードでは、反応座標に対する時系列データ $\xi(t)$ から、反応座標 ζ に沿った自由エネルギープロファイル $W(\zeta)$ を計算する。 $W(\zeta)$ は平均力ポテンシャルとも呼ばれる。

4.4.1 理論背景

系の配置を $\mathbf{X}$ 、そしてポテンシャルを $U(\mathbf{X})$ とする。このとき、配置積分 Z は逆温度を β として

$$Z = \int d\mathbf{X} \exp(-\beta U(\mathbf{X})) \quad (4.4)$$

で与えられる。同様に、系が反応座標空間 ($\zeta(\mathbf{X})$) 上の微小区間 $\zeta - \Delta\zeta$ にトラップされている状態に対応する分配関数は

$$Z(\zeta) = \Delta\zeta \int d\mathbf{X} \delta(\zeta - \xi(\mathbf{X})) \exp(-\beta U(\mathbf{X})) \quad (4.5)$$

である。 ζ に対する確率密度を

$$\rho(\zeta) = \langle \delta(\zeta - \xi(\mathbf{X})) \rangle = \frac{1}{Z} \int d\mathbf{X} \delta(\zeta - \xi(\mathbf{X})) \exp(-\beta U(\mathbf{X})) \quad (4.6)$$

と定義すると、

$$\frac{Z(\zeta)}{Z} = \rho(\zeta) \Delta\zeta \quad (4.7)$$

が成り立つ。したがって、系が領域 $\zeta - \Delta\zeta$ に制限されている状態と制限されていない状態の自由エネルギー差 $\Delta F(\zeta)$ は

$$\Delta F(\zeta) = -\frac{1}{\beta} \log \frac{Z(\zeta)}{Z} = -\frac{1}{\beta} \log (\rho(\zeta) \Delta\zeta) \quad (4.8)$$

と表せる。 ζ_0 を基準にとった自由エネルギーは

$$\begin{aligned} W(\zeta) &= \Delta F(\zeta) - \Delta F(\zeta_0) \\ &= -\frac{1}{\beta} \log \frac{\rho(\zeta)}{\rho(\zeta_0)} \end{aligned} \quad (4.9)$$

である。この W のことを平均力ポテンシャルと呼ぶ。 ζ_0 の設定は任意であるが、多くの場合、 $W(\zeta)$ の最小値が 0 となるようにとる。 $W(\zeta)$ を計算することで、 ζ に応じた熱力学的な安定性の変化を議論することが可能となる。式 (4.9) は、 ζ に依存する部分と定数 C に分けて

$$W(\zeta) = -\frac{1}{\beta} \log \int d\mathbf{X} \delta(\zeta - \xi(\mathbf{X})) \exp(-\beta U(\mathbf{X})) + C \quad (4.10)$$

と書き直すこともできる。ただし、 C は

$$C = \frac{1}{\beta} \log \int d\mathbf{X} \delta(\zeta_0 - \xi(\mathbf{X})) \exp(-\beta U(\mathbf{X})) \quad (4.11)$$

で与えられる。

4.4.2 パラメータ一覧

Namelist	パラメータ	備考
&input_param	flist_cv	CV ファイル一覧を格納したファイル名
	flist_weight	重みファイル一覧を格納したファイル名 (オプション)
	fcv	CV ファイル名
	flist_weight	重みファイル名 (オプション)
&output_param	fhead	出力ファイル名のヘッダ
&option_param	ndim	CV の次元数
	xyzcol	CV データ中の列番号
	ng3	各 CV 方向のグリッド数
	del	各 CV 方向のグリッド間隔
	origin	各 CV 方向のグリッド原点
	temperature	温度 [K]
	use_radial	動径成分の CV を含むかどうか
	radcol	CV データ中にある動径成分の列番号
	box_system	系のボックスサイズ (ndim=3 の場合のみ使用)
	use_spline	スプライン補間の有無
	spline_resolution	スプライン補間の分解能
	gen_script_mpl2d	2 次元 PMF 描画用 Python スクリプト生成の有無
	skip_calc	PMF 計算をスキップするかどうか
&matplotlib_param	fpdf	出力 PDF ファイル名
	labels	軸ラベルおよびカラーバーラベル
	ranges	描画範囲
	tics	軸目盛り間隔
	scales	軸の単位変換係数

4.4.3 &option_param

- **ndim**

- Default: None
- Example: ndim = 2

Collective Variable (CV) の次元数を指定する。例えば ndim = 2 の場合は 2 次元 PMF, ndim = 3 の場合は 3 次元 PMF を計算する。

- **xyzcol**

- Default: 1
- Example: `xyzcol = 1 2`

CV データファイル中で、各 CV に対応する列番号を指定する。先頭列は時刻やステップ数として扱われるため無視される。

例えば

```
time  cv1  cv2  cv3
```

の形式でデータが格納されている場合、

```
xyzcol = 1 2
```

とすると、`cv1` と `cv2` を用いて 2 次元 PMF を計算する。

- **ng3**

- Default: 1 1 1
- Example: `ng3 = 10 20`

各 CV 方向のグリッド数を指定する。

`ndim = N` の場合、`ng3` には各次元に対するグリッド数を順番に与える。

- **del**

- Default: 1.0 1.0 1.0
- Example: `del = 0.1d0 0.2d0`

各 CV 方向のグリッド間隔を指定する。

i 番目の CV について、グリッド間隔を Δ_i とすると、隣接するグリッド点の距離は Δ_i となる。

- **origin**

- Default: 0.0 0.0 0.0
- Example: `origin = 0.0d0 1.0d0`

各 CV 方向のグリッドの原点を指定する。

k 番目の CV について、原点を $x_k^{(0)}$ 、グリッド間隔を Δ_k とすると、 i 番目のグリッド位置は

$$x_k^{(i)} = x_k^{(0)} + (i - 1)\Delta_k \quad (4.12)$$

で与えられる。

- **temperature**

- Default: 300.0d0
- Example: `temperature = 298.0d0`

PMF 計算で用いる温度 T [K] を指定する.

- **use_radial**

- Default: `.false.`
- Example: `use_radial = .true.`

CV が動径成分であるかどうかを指定する. `.true.` の場合には, 次の `radcol` で指定した CV 成分 x_k について, ビンあたりの体積を $4\pi x_k^2 \Delta_k^2$ として分布関数を計算する.

- **radcol**

- Default: N/A
- Example: `radcol = 1`

何番目の CV が動径成分であるかを指定する. `use_radial = .true.` の場合のみ有効となる.

- **box_system**

- Default: 0.0 0.0 0.0
- Example: `box_system = 50.0d0 50.0d0 50.0d0`

系のボックスサイズを指定する.

このパラメータは `ndim = 3` の場合に使用される. 各方向のボックス長を順番に与える.

- **use_spline**

- Default: `.false.`
- Example: `use_spline = .true.`

PMF 計算後にスプライン補間を行うかどうかを指定する.

`.true.` の場合はスプライン補間を実施し, `.false.` の場合は元のグリッドをそのまま使用する.

- **spline_resolution**

- Default: 4

- Example: `spline_resolution = 8`

スプライン補間時の分解能を指定する。

大きな値を指定するほど細かい補間グリッドが生成される。

- **gen_script_mpl2d**

- Default: `.false.`

- Example: `gen_script_mpl2d = .true.`

2次元PMFを可視化するためのPythonスクリプトを生成するかどうかを指定する。

現在は `ndim = 2` の場合のみサポートされている。

- **skip_calc**

- Default: `.false.`

- Example: `skip_calc = .true.`

PMF計算をスキップするかどうかを指定する。

`.true.` の場合はPMF計算を行わず、補助ファイルの生成などのみを実行する。`.false.` の場合は通常通りPMF計算を実行する。このオプションは、Matplotlib (Python) スクリプトを設定変した上で再作成したい場合に便利である。

4.4.4 &matplotlib_param

Python (Matplotlib) で2次元PMFを可視化するためのスクリプト作成するためのオプション。&option_param内で `gen_script_mpl2d = .true.` とした場合に有効となる。

- **fpdf**

- Default: `None`

- Example: `fpdf = "pmf.pdf"`

出力するPDFファイル名を指定する。

- **labels**

- Default: `None`

- Example: `labels = "$x$" "$y$" "PMF"`

プロットに表示する軸ラベルを指定する。

`ndim = 2` の場合は、

```
labels = "xlabel" "ylabel" "zlabel"
```

の順に, x 軸, y 軸, および PMF 値に対応するカラーバーのラベルを指定する.

LaTeX 形式の数式表記を利用できる.

• ranges

- Default: None
- Example: `ranges = -10.0 10.0 -5.0 5.0 0.0 10.0`

描画範囲を指定する.

`ndim = 2` の場合は,

```
ranges = xmin xmax ymin ymax zmin zmax
```

の順に指定する.

ここで,

- `xmin, xmax`: x 軸の表示範囲
- `ymin, ymax`: y 軸の表示範囲
- `zmin, zmax`: PMF 値の表示範囲

を表す.

• tics

- Default: None
- Example: `tics = 1.0 1.0 0.5`

軸目盛りの間隔を指定する.

`ndim = 2` の場合は,

```
tics = xtic ytic ztic
```

の順に指定する.

ここで,

- `xtic`: x 軸の目盛り間隔
- `ytic`: y 軸の目盛り間隔
- `ztic`: PMF 値に対応するカラーバーの目盛り間隔

を表す.

- **scales**

- Default: 1.0 1.0 1.0
- Example: scales = 10.0 10.0 0.239006

軸の単位変換係数を指定する.

ndim = 2 の場合は,

```
scales = xscale yscale zscale
```

の順に指定する.

読み込まれたデータ値は描画時に各係数倍される.

例えば,

```
scales = 10.0 10.0 1.0
```

とすると, x 軸および y 軸の値は 10 倍された状態で表示される.

4.4.5 出力ファイルのフォーマット

- *_original.gnplt

このファイルに PMF ($W(\zeta)$) が出力される.

- 1 列目: 座標
- 2 列目: PMF

- *_g_original.gnplt

このファイルに分布関数 ($\rho(\zeta) / \rho_0(\zeta_0)$) が出力される.

- 1 列目: 座標
- 2 列目: 分布関数

use_spline = .true. の場合には, スプライン後の PMF と分布関数がそれぞれ *_spline.gnplt, *_g_spline.gnplt という名前で出力される.

4.4.6 解析例

1つのデータ列を持つ時系列データ `inp.ts` から, $5 \leq x \leq 10$ の範囲について $\Delta x = 0.1$ の間隔で PMF ($W(x)$) を計算する例を以下に示す.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
  fcv      = "inp.ts"
/
&output_param
  fhead = "out"
/
&option_param
  ndim      = 1
  xyzcol    = 1
  ng3       = 50
  del       = 0.1
  origin    = 5.0
  temperature = 300.0d0
  use_spline = .false.
  use_radial = .false.
/
EOF

fanatra potential_of_mean_force run.inp
```

4.5 制限つき動径平均力ポテンシャル (restricted_radial, rrpmpf)

この解析モードでは, 何らかの反応座標 (集団座標) によって条件づけられた状態の動径方向に対する自由エネルギープロファイルを計算できる. また, このプロファイルから, ホスト分子とゲスト分子の結合に伴う自由エネルギー変化 (結合自由エネルギー) の計算が可能である.

4.5.1 理論背景

系が配置 $\mathbf{X}$ をとるときのホスト-ゲスト分子間の距離, 結合状態を規定する反応座標の値をそれぞれ $d(\mathbf{X})$, $\xi(\mathbf{X})$ とする. 距離に対する平均力ポテンシャルを

$$w(r) = -\frac{1}{\beta} \log \left[\int d\mathbf{X} \frac{\delta(r - d(\mathbf{X}))}{4\pi r^2} \exp(-\beta U(\mathbf{X})) \right] + C \quad (4.13)$$

と定義する. さらに, 反応座標空間上の結合状態に対応する領域を Υ とし, 次式に示す特性関数 $\Theta(\xi)$ を導入する.

$$\Theta_B(\xi) = \begin{cases} 1 & \xi \in \Upsilon \\ 0 & \xi \notin \Upsilon \end{cases} \quad (4.14)$$

この $\Theta(\xi)$ を用いれば, 結合状態に属する配置に対する平均力ポテンシャルは

$$w_B(r) = -\frac{1}{\beta} \log \left[\int d\mathbf{X} \frac{\delta(r - d(\mathbf{X}))}{4\pi r^2} \Theta_B(\xi(\mathbf{X})) \exp(-\beta U(\mathbf{X})) \right] + C \quad (4.15)$$

と表せる. $w(r)$ と $w_B(r)$ に含まれる定数 C は同一の値とし, $w_B(r)$ が最小値がゼロとなるように設定する. $w(r)$ は r が十分に大きいときに定数となるので, その定数を Δw とし, $w(r) = \Delta w$ を満たす領域 U を $r_0 \leq r \leq r_1$ と定める. r_0, r_1 の設定は, $w(r) = \Delta w$ を満たしてさえいれば, 任意に決めて良い. このことについては後で述べる. 2 分子が $r_0 \leq r \leq r_1$ の距離範囲に存在する状態 U と結合状態 B の存在確率比は

$$\frac{\int_{\Upsilon} dr \exp(-\beta w_B(r))}{\int_{r_0}^{r_1} dr \exp(-\beta w(r))} \quad (4.16)$$

であることから, U から B の変化に伴う自由エネルギー変化 ΔG_{PMF} は

$$\Delta G_{\text{PMF}} = -\frac{1}{\beta} \log \frac{\int_{\Upsilon} dr 4\pi r^2 \exp(-\beta w_B(r))}{\int_{r_0}^{r_1} dr 4\pi r^2 \exp(-\beta w(r))} \quad (4.17)$$

と表せる. 分母に含まれる $w(r)$ は $r_0 \leq r \leq r_1$ の範囲で Δw なので,

$$\Delta G_{\text{PMF}} = -\Delta w - \frac{1}{\beta} \log \frac{V_B}{V_U} \quad (4.18)$$

と書き直せる. ただし, V_B, V_U は

$$V_B = \int_{\Upsilon} dr 4\pi r^2 \exp(-\beta w_B(r)) \quad (4.19)$$

$$V_U = \int_{r_0}^{r_1} dr 4\pi r^2 \quad (4.20)$$

で定義される. 式 (4.18) の第 2 項は, ゲスト分子が濃度 $1/V_U$ の状態から濃度 $1/V_B$ の状態への移行に伴う自由エネルギー変化である. 実験での結合自由エネルギーは標準状態濃度 (典型的には $c^\circ = 1 \text{ mol L}^{-1}$) のゲスト分子がホスト分子に結合するのに伴う自由エネルギー変化 (ΔG°) であり, ΔG_{PMF} とは異なる. ゲスト分子 1 つで濃度 c° となるときの体積を V° とするとこれは $V^\circ = 1661 \text{ \AA}^3$ である. ゲスト分子が濃度 $c^\circ = 1/V^\circ$ の状態から濃度が $1/V_U$ の状態への移行に対する自由エネルギー変化 $\Delta G_{\text{corr}}^\circ$ は

$$\Delta G_{\text{corr}}^\circ = -\frac{1}{\beta} \log \frac{V_U}{V^\circ} \quad (4.21)$$

である。つまり、 $1/V^\circ \rightarrow 1/V_B$ の変化を $1/V^\circ \rightarrow 1/V_U$ と $1/V_U \rightarrow 1/V_B$ の 2 段階に分けることで、

$$\Delta G^\circ = \Delta G_{\text{PMF}} + \Delta G_{\text{corr}}^\circ \quad (4.22)$$

と表すことができる。式 (4.18), (4.21) を上式に代入することで、

$$\Delta G^\circ = -\Delta w - \frac{1}{\beta} \log \frac{V_B}{V^\circ} \quad (4.23)$$

を得る。この式には V_U が含まれていないことから、 ΔG° の値は r_0, r_1 の選択に依存しない。

4.5.2 パラメーター一覧

Namelist	パラメータ	備考
&input_param	flist_cv	CV ファイル一覧を格納したファイル名
	flist_weight	重みファイル一覧を格納したファイル名 (オプション)
	fcv	CV ファイル名
	fweight	重みファイル名
&output_param	fhead	出力ファイル名のヘッダ
&option_param	calcfe	標準自由エネルギーを計算するかどうか
	ndim	CV の次元数
	only_r	分子間距離が唯一の CV の場合かどうか
	temperature	温度 [K]
	dr	距離グリッド間隔
	ngrid	距離グリッド数
	nsta	解析開始フレーム番号
	use_bootstrap	ブートストラップ解析の有無
	vol0	標準状態体積
	urange	非結合状態の距離範囲
&bootopt_param	bound_range	結合状態の距離範囲
	iseed	乱数シード
	nsample	各試行で抽出するサンプル数
	duplicate	重複抽出を許可するかどうか
	ntrial	ブートストラップ試行回数

4.5.3 入力ファイル (時系列データ) のフォーマット

- 1 列目：時間軸
- 2 列目～($N + 1$) 列目：CV 値
- 最終列：分子間距離

1 列目は読み込まれず、2 列目から 1 つめのデータ列としてカウントされる。分子間距離のみが CV の場合、つまり

- 1 列目：時間軸
- 2 列目：分子間距離

の場合には、`&option_param` 内で `only_r = .true.` を指定すれば良い。

4.5.4 `&option_param`

- **calcfe**
 - Default: `.true.`
 - Example: `calcfe = .true.`

標準結合自由エネルギーを計算するかどうかを指定する。`.true.` の場合は標準結合自由エネルギーを計算し、`.false.` の場合は PMF のみを出力する。

- **ndim**
 - Default: `None`
 - Example: `ndim = 1`

CV の次元数を指定する。現在は 1 次元および多次元 PMF に対応している。

- **only_r**
 - Default: `.false.`
 - Example: `only_r = .false.`

分子間距離が CV であり、それ以外の CV が存在しない場合に `.true.` を指定する。

- **temperature**
 - Default: `298.0`
 - Example: `temperature = 300.0`

自由エネルギー計算に用いる温度 T [K] を指定する。

- **dr**

- Default: None
- Example: `dr = 0.1d0`

距離方向のグリッド間隔を指定する。グリッド点間の間隔 Δr として用いられる。

- **ngrid**

- Default: None
- Example: `ngrid = 1000`

距離方向のグリッド数を指定する。PMF は `ngrid` 個のグリッド上で評価される。

- **nsta**

- Default: 1
- Example: `nsta = 100`

解析を開始するフレーム番号を指定する。`nsta` より前のフレームは解析から除外される。

- **use_bootstrap**

- Default: `.false.`
- Example: `use_bootstrap = .true.`

ブートストラップ法による誤差評価を実施するかどうかを指定する。`.true.` の場合は `&bootopt_param` の設定に従ってブートストラップ解析を行う。

- **vol0**

- Default: `1661.0d0`
- Example: `vol0 = 1661.0d0`

標準状態体積 V° を指定する。デフォルト値は Å 単位で 1 mol/L に対応する体積 (1661 Å^3) である。

- **urange**

- Default: None
- Example: `urange = 25.0 30.0`

非結合状態 (unbound state) とみなす距離範囲を指定する。1 つ目の値が下限, 2 つ目の値が上限を表す。

- **bound_range**

- Default: None

- Example:

```
react_range = 0.0 10.0
              5.0 20.0
```

結合状態 (bound state) の CV 範囲を指定する。各行について、1 つ目の値が下限、2 つ目の値が上限を表す。ndim 個の範囲を指定することで、多次元結合領域を定義できる。

4.5.5 &bootopt_param

ここでは、ブートストラップ法による誤差解析の設定を行う。use_bootstrap = .true. の場合に有効となる。

- **iseed**

- Default: 自動生成

- Example: iseed = 3141592

ブートストラップ解析で使用する乱数シードを指定する。iseed <= 0 の場合はプログラムが自動的にシードを生成する。

- **nsample**

- Default: None

- Example: nsample = 1000

各ブートストラップ試行で抽出するサンプル数を指定する。

- **duplicate**

- Default: .true.

- Example: duplicate = .false.

ブートストラップサンプリング時に重複抽出を許可するかどうかを指定する。

- **ntrial**

- Default: None

- Example: ntrial = 100

ブートストラップ試行回数を指定する。試行回数を増やすことで統計誤差の推定精度が向上するが、計算コストも増加する。

4.5.6 出力ファイルのフォーマット

- *.rd
 - 1 列目: 分子間距離 r
 - 2 列目: r に沿った通常の PMF
 - 3 列目: 結合状態に対応する PMF

4.5.7 解析例

CV 数が 1, 最終列に分子間距離を格納した時系列データ `inp.ts` から, $18 \leq r/\text{\AA} \leq 20$ の範囲を解離状態, 結合状態に対応する CV (ζ) の値域が $5.0 \leq \zeta \leq 10.0$ の範囲のときの結合自由エネルギーの値を計算する例を下記に記す.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
  fcv      = "inp.ts"
/
&output_param
  fhead    = "out"
/
&option_param
  calcfe   = .true.
  only_r   = .false.
  ndim     = 1
  temperature = 300.0
  dr       = 0.1d0
  ngrid    = 1000
  nsta     = 1
  use_bootstrap = .false.
  vol0     = 1661.0d0
  urange   = 18.0 20.0
  bound_range = 5.0 10.0
/
EOF

fanatra restricted_radial run.inp
```

これを実行すると, ログファイルの後半に結果が出力される.

```
... Thermodynamic analysis ...
vb      (A^3)      =      21.5444908
wu      (kcal/mol) =      -1.3486059
wb      (kcal/mol) =       0.0000000
dw      (kcal/mol) =      -1.3486059
dGv     (kcal/mol) =       2.5903536
dGzero  (kcal/mol) =       3.9389595
Keq     (M^-1)     =      1.3506025E-03
```

4.6 フレーム抽出 (extract_frame, extf)

この解析モードでは、CV の値域指定により、特定の条件を満たすスナップショット (フレーム) を抽出する。

4.6.1 パラメーター一覧

Namelist	パラメータ	備考
&input_param	flist_traj	トラジェクトリファイル一覧を格納したファイル名
	flist_cv	CV ファイル一覧を格納したファイル名
	ftraj	トラジェクトリファイル名
	fcv	CV ファイル名
&output_param	fhead	出力ファイル名のヘッダ
&option_param	dt	トラジェクトリの時間刻み
	ndim	CV の次元数
	extract_range	フレーム抽出を行う CV 範囲

4.6.2 &option_param

- **ndim**

- Default: None
- Example: ndim = 2

時系列 CV データの次元数を指定する。

extract_range には、この次元数に対応した範囲を指定する必要がある。

- **extract_range**

- Default: None
- Example:

```
extract_range = -5.0d0 5.0d0
               1.0d0 0.0d0
```

フレームを抽出する CV 領域を指定する.

各 CV について, 下限値と上限値を 1 組として与える.

上記の例では,

- CV1: $-5.0 \leq CV_1 \leq 5.0$
- CV2: $0.0 \leq CV_2 \leq 1.0$

を満たすフレームが抽出される.

ndim 個の範囲を指定することで, 多次元の抽出条件を設定できる.

4.6.3 出力ファイルのフォーマット

出力ファイル(トラジェクトリ)のフォーマットは, 入力のトラジェクトリのフォーマット (dcd or xtc or nc (netcdf)) と同じものとなる.

4.6.4 解析例

CV 数が 1, CV (ζ) の時系列データ `inp.ts` をもとに, $5 \leq \zeta \leq 10$ の条件を満たすフレームを入力トラジェクトリから抽出する場合の解析例を示す.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
  ftraj    = "input.dcd"
  fcv      = "inp.ts"
/
&output_param
  fhead    = "out"
/
&option_param
  ndim      = 1
  extract_range = 5.0 10.0
/
EOF
```

(次のページに続く)

```
fanatra extract_frame run.inp
```

これを実行すると、out.dcd が出力される。

4.7 フレームのランダム抽出 (random_frame, randf)

この解析モードでは、ランダムにスナップショット (フレーム) を抽出する。

4.7.1 パラメータ一覧

Namelist	パラメータ	備考
&input_param	ftraj	入力トラジェクトリファイル名
	flist_traj	トラジェクトリファイル一覧を格納したファイル名
&output_param	fhead	出力ファイル名のヘッダ
&option_param	nsample	ランダム抽出するフレーム数
	iseed	乱数シード
	output_trajtype	出力トラジェクトリ形式
	duplicate	重複抽出を許可するかどうか
	out_rst7	AMBER rst7 形式の出力有無
	shuffle	フレームをシャッフルするかどうか

4.7.2 &option_param

- **nsample**

- Default: 100
- Example: nsample = 100

ランダムに抽出するフレーム数を指定する。

- **iseed**

- Default: -1 (自動生成)
- Example: iseed = 3141592

ランダム抽出に使用する乱数シードを指定する.

`iseed <= 0` の場合はプログラムが自動的にシードを生成する.

- **output_trajtype**

- Default: `dcd`
- Example: `output_trajtype = "dcd"`

出力するトラジェクトリ形式を指定する.

現在は

- `dcd`
- `xtc`
- `netcdf`

を指定できる.

- **duplicate**

- Default: `.false.`
- Example: `duplicate = .true.`

同じフレームを複数回抽出することを許可するかどうかを指定する.

- **out_rst7**

- Default: `.false.`
- Example: `out_rst7 = .true.`

抽出した各フレームを AMBER `rst7` 形式でも出力するかどうかを指定する.

- **shuffle**

- Default: `.false.`
- Example: `shuffle = .true.`

フレームをシャッフルして出力するかどうかを指定する. `.true.` の場合は全フレームをランダムな順序に並べ替えて出力する.

4.7.3 出力ファイルのフォーマット

出力ファイル(トラジェクトリ)のフォーマットは, 入力のトラジェクトリのフォーマット (dcd or xtc or nc (netcdf)) と同じものとなる.

4.7.4 解析例

トラジェクトリファイルとして, `input.dcd` を指定し, ランダムに 100 フレームを抽出する場合の例を以下に示す. また, トラジェクトリと合わせて `rst7` 形式 (`inpcrd`) で各フレームでの構造を出力する.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
  ftraj      = "input.dcd"
/
&output_param
  fhead      = "out"
/
&option_param
  nsample = 100
  output_trajtype = "dcd"
  duplicate      = .false.
  out_rst7       = .true.
/
EOF

fanatra random_frame run.inp
```

これを実行すると, `out.dcd` と `outXXXXX.inpcrd` (`XXXXX=00001, ..., 00100`) が出力される.

4.8 移動平均(moving_average, movave)

この解析モードでは, 与えられた 1 次元データ (分布関数など) の移動平均による平滑化を行う.

4.8.1 パラメータ一覧

Namelist	パラメータ	備考
&input_param	fcv	関数データファイル名
	flist_cv	関数データファイル一覧を格納したファイル名
&output_param	fhead	出力ファイル名のヘッダ
&option_param	dx	グリッド間隔
	xsta	グリッドの開始位置
	xsep	領域の境界位置
	nregion	領域数
	npoint	各領域で移動平均に用いる点数
	include_zero	原点を保持するかどうか

4.8.2 &option_param

- **dx**

- Default: None
- Example: dx = 0.1d0

関数データのグリッド間隔を指定する.

- **xsta**

- Default: 0.0d0
- Example: xsta = 0.0d0

グリッドの開始位置を指定する.

i 番目のグリッド位置を x_i とすると,

$$x_i = x_0 + (i - 1)\Delta x \quad (4.24)$$

で与えられる.

- **nregion**

- Default: 1
- Example: nregion = 2

異なる移動平均条件を適用する領域数を指定する. npoint には, この領域数に対応した個数の値を指定する.

- **xsep**

- Default: `0.0d0`
- Example: `xsep = 1.0d0`

異なる移動平均幅を適用する領域の境界を指定する. `nregion = 1` の場合は使用されず, `xsep = 0.0d0` を指定すればよい. 例えば, N 領域に分ける場合には, 境界値を $N - 1$ 個指定する必要がある.

- **npoint**

- Default: `None`
- Example: `npoint = 3 5`

各領域で移動平均に用いるデータ点数を指定する. 奇数を指定する必要がある. 上記の例では,

- 第 1 領域: 3 点移動平均
- 第 2 領域: 5 点移動平均

を適用する.

- **include_zero**

- Default: `.true.`
- Example: `include_zero = .false.`

移動平均後に原点での値を保持するかどうかを指定する. `.true.` の場合は, 原点での値は入力値のまま固定される.

4.8.3 出力ファイルのフォーマット

- ***XXXX.dat**

- 1 列目: x 軸
- 2 列目: 関数値

`flist_cv` などを入力したファイルごとに平滑化されたファイルが出力される.

4.8.4 解析例

1次元の関数 $f(x)$ を格納したファイル `input.dat` (軸の始点 `0`, グリッド間隔 `0.1`) を $x \leq 1.0, x > 1.0$ の2つの領域に分割して, それぞれの領域について3点平均, 5点平均する場合のインプットを下記に示す.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
  fcv      = "input.dat"
/
&output_param
  fhead = "out"
/
&option_param
  dx      = 0.1d0
  xsta    = 0.0d0
  xsep    = 1.0d0
  nregion = 2
  npoint  = 3 5
  include_zero = .true.
/
EOF

fanatra moving_average run.inp
```

これを実行すると, 平滑化された関数データを格納した `out0001.dat` が出力される.

4.9 関数の平均 (average_function, avef)

この解析モードでは, 複数の入力ファイルに格納されている1次元関数を平均する. 例えば, i 番目のファイルに格納されている関数を $f_i(x)$, ファイル数を N とすると, 平均後の関数 $f_{\text{ave}}(x)$ は

$$f_{\text{ave}}(x) = \frac{1}{N} \sum_{i=1}^N f_i(x), \quad (4.25)$$

で表される.

4.9.1 パラメーター一覧

Namelist	パラメータ	備考
&input_param	flist_cv	関数データファイル一覧を格納したファイル名
&output_param	fhead	出力ファイル名のヘッダ
&option_param	use_bootstrap	ブートストラップ解析の有無
	xsta	グリッドの開始位置
	dx	グリッド間隔
&bootopt_param	duplicate	重複抽出を許可するかどうか
	iseed	乱数シード
	nsample	各試行で抽出するサンプル数
	ntrial	ブートストラップ試行回数

4.9.2 &option_param

- **use_bootstrap**

- Default: `.false.`
- Example: `use_bootstrap = .true.`

ブートストラップ法による誤差評価を実施するかどうかを指定する。 `.true.` の場合は、 `&bootopt_param` の設定に従ってブートストラップ解析を行う。

- **xsta**

- Default: `0.0d0`
- Example: `xsta = 0.0d0`

グリッドの開始位置を指定する。

i 番目のグリッド位置を x_i とすると、

$$x_i = x_0 + (i - 1)\Delta x \quad (4.26)$$

で与えられる。

- **dx**

- Default: `0.1d0`
- Example: `dx = 0.1d0`

関数データのグリッド間隔を指定する。

4.9.3 `&bootopt_param`

ここでは、ブートストラップ法による誤差解析の設定を行う。 `use_bootstrap = .true.` の場合に有効となる。

- **iseed**

- Default: 自動生成
- Example: `iseed = 3141592`

ブートストラップ解析で使用する乱数シードを指定する。 `iseed <= 0` の場合はプログラムが自動的にシードを生成する。

- **nsample**

- Default: None
- Example: `nsample = 1000`

各ブートストラップ試行で抽出するサンプル数を指定する。

- **duplicate**

- Default: `.true.`
- Example: `duplicate = .false.`

ブートストラップサンプリング時に重複抽出を許可するかどうかを指定する。

- **ntrial**

- Default: None
- Example: `ntrial = 100`

ブートストラップ試行回数を指定する。試行回数を増やすことで統計誤差の推定精度が向上するが、計算コストも増加する。

4.9.4 出力ファイルのフォーマット

- ***.ave**

- 1 列目: x 軸
- 2 列目: 関数値
- 3 列目: 標準誤差 (`use_bootstrap = .true.` の場合)

4.9.5 解析例

1次元の関数 $f(x)$ (軸の始点 0 , グリッド間隔 0.1) がファイルごとに格納されていて, そのファイルリストが `list` にまとめられているとして, その平均と標準誤差をブートストラップ法で計算する例を示す.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
  flist_cv = "list"
/
&output_param
  fhead      = "out"
/
&option_param
  use_bootstrap = .false.
  xsta         = 0.0d0
  dx           = 0.1d0
/

&bootopt_param
  duplicate    = .true.
  iseed        = 3141592
  nsample      = 1000
  ntrial       = 100
/
EOF

fanatra average_function run.inp
```

これを実行すると, 平均化された関数データを格納した `out.ave` が出力される.

第5章 Integral-Equation formalism of Population DYNamics (IEPDYN)

Integral-Equation formalism of Population DYNamics (IEPDYN) 法は、状態間遷移を記述する方法論である。この方法の特徴は、連続的な時間変化によって状態ダイナミクスを表現する点にあり、Markov-state model (MSM) をはじめとする既存理論で導入される離散的な時間スケール (ラグタイム) を必要とせず、ラグタイムに依存しないダイナミクスの記述が可能である。また、IEPDYN 法による記述には局所的な状態間遷移に関する時間相関関数の計算が必要となるが、これらは局所的な情報のみを含むため、短時間 MD 計算から評価することができる。したがって、短時間 MD 計算から得られる情報を用いて、長時間スケールにわたるポピュレーションダイナミクスを記述することが可能である。

ANATRA では、fanatra の解析モードの一つとして IEPDYN 法を実装している。

5.1 理論背景

5.1.1 IEPDYN の理論表式

注目系を構成する原子の位置と運動量のセット (位相点) を Γ とし、 Γ で張られた空間 (位相空間) を複数の構造状態に分割する。系の Liouville 演算子を $\mathcal{L}$ とすると、時刻 t における状態 j の確率密度 $\hat{f}_j(\Gamma, t)$ の厳密な時間発展は次式によって表される。

$$\frac{\partial}{\partial t} \hat{f}_j(\Gamma, t) = -\mathcal{L} \hat{f}_j(\Gamma, t) - \sum_{i \in \mathcal{N}_j} S_{ij}(\Gamma) \hat{f}_j(\Gamma, t) + \sum_{k \in \mathcal{N}_j} S_{jk}(\Gamma) \hat{f}_k(\Gamma, t). \quad (5.1)$$

ここで、 $\mathcal{N}_j$ は状態 j の隣接状態のセット、 S_{ij} は反応シンク関数と呼ばれ、 $S_{ij}(\Gamma) \hat{f}_j(\Gamma, t)$ が時刻 $t \sim t + dt$ の間に系が状態 j から i に遷移する確率となるように定義される。時刻 t において系が状態 j として存在する確率 $P_j(t)$ は、 $\hat{f}_j(\Gamma, t)$ を Γ で積分することにより得られる。上式の形式解から $P_j(t)$ の近似表式

$$P_j(t) = P_j^0(t) + \sum_{k \in \mathcal{N}_j} \int_0^t d\tau M_{jk}(t - \tau) Q_{jk}(\tau), \quad (5.2)$$

が得られる。式中の $P_j^0(t)$ は時刻 0 で状態 j の系が時刻 t でも隣接状態に一度も遷移せずに滞留する確率、 $Q_{jk}(t)$ は時刻 $t \sim t + dt$ で系が状態 k から j に遷移する確率、そして、 $M_{jk}(t)$ は状態 k から j に遷移した時刻を 0 とし、時刻 t でも隣接状態に遷移することなく状態 j に滞留する確率である。 $P_j^0(t)$ や $M_{jk}(t)$ は単一の遷移イベントしか関与しないため、短時間 MD により計算可能である。さらに、 $Q_{jk}(t)$ についても、短時間スケールで減衰する関数を積分核に持つ積分方程式

$$Q_{ij}(t) = R_{ij}(t) + \sum_{k \in \mathcal{N}_j} \int_0^t d\tau K_{ijk}(t - \tau) Q_{jk}(\tau), \quad (5.3)$$

で表せることが分かっている．ここで， $R_{ij}(t)$ は時刻 0 で状態 j にいた系が時刻 $t \sim t + dt$ の間に状態 i に遷移する確率， $K_{ijk}(t)$ は時刻 0 $\sim dt$ の間に状態 k から j に遷移した系が，時刻 $t \sim t + dt$ で状態 i に遷移する確率を表す． $P_j^0(t)$, $M_{jk}(t)$ と同様に， $R_{ij}(t)$, $K_{ijk}(t)$ は短時間 MD から計算可能である．さらに， $R_{ij}(t)$, $K_{ijk}(t)$ から $P_j^0(t)$, $M_{jk}(t)$ を構成することが可能である．したがって，短時間 MD を実行し $R_{ij}(t)$, $K_{ijk}(t)$ を計算することで，長時間スケールのポピュレーションダイナミクス ($P_j(t)$) の予測が可能である．

IEPDYN 法によるポピュレーションダイナミクスの記述では，初期時刻における各状態のポピュレーション w_j を与える必要がある．初期時刻で系が平衡にあるとした場合，初期時刻でポピュレーションを持つ状態のセットを $\mathcal{M}_I$ ，状態 j の平衡ポピュレーションを P_j^{eq} とすると， w_j は

$$\forall j \in \mathcal{M}_I, w_j = \frac{P_j^{\text{eq}}}{\sum_{k \in \mathcal{M}_I} P_k^{\text{eq}}} \quad (5.4)$$

で与えられる． w_j はアンブレラサンプリング法などによる自由エネルギー計算によって求めても良いが，後で示すように，IEPDYN 法で平衡ポピュレーションを解析的評価することも可能である．

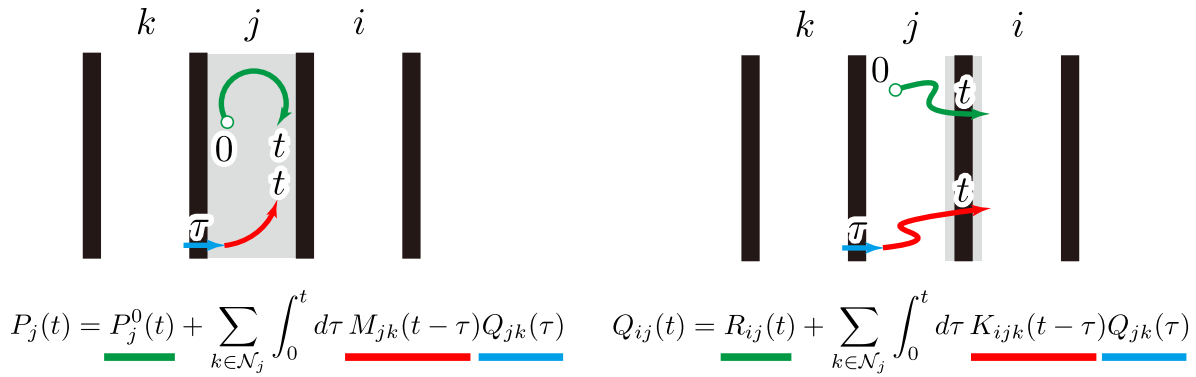


図 5.1: IEPDYN 法のスキーム図．

5.1.2 境界条件の導入

IEPDYN 法では，粗視化された反応ダイナミクスに対して，反射境界条件 (reflecting boundary condition) および吸収境界条件 (absorbing boundary condition) を導入することができる．これらの境界条件は，状態間遷移確率 $Q_{ij}(t)$ に対して直接課され，再帰確率 (RP) 理論を用いた分子結合過程の解析に利用される．

反射境界条件では，ある状態へ流入したポピュレーションは，その状態に滞留することなく直ちに隣接状態へ再分配される．流入を許さない状態の集合を $\mathcal{M}_{\text{reflect}}$ とすると，反射境界条件は

$$\forall i \in \mathcal{M}_{\text{reflect}}, \forall j \in \mathcal{N}_i, \quad Q_{ji}(t) = R_{ij}(t) + \sum_{k \in \mathcal{N}_j} \int_0^t d\tau K_{ijk}(t-\tau) Q_{jk}(\tau), \quad (5.5)$$

$$Q_{ij}(t) = 0, \quad (5.6)$$

によって与えられる．上式は，状態 j から $i \in \mathcal{M}_{\text{reflect}}$ へ流入したポピュレーションが即座に状態 j へ折り返されることを表している．

一方、吸収境界条件では、ある状態へ到達したポピュレーションは、その後他の状態へ遷移しないものとする。吸収状態の集合を $\mathcal{M}_{\text{absorb}}$ とすると、吸収境界条件は

$$\forall i \in \mathcal{M}_{\text{absorb}}, \forall j \in \mathcal{N}_i, \quad Q_{ji}(t) = 0, \quad (5.7)$$

によって与えられる。この条件は、吸収状態から他の状態への流入が完全に存在しないことを意味しており、一度吸収状態へ到達したポピュレーションは終状態として取り扱われる。

このように、IEPDYN 法では、状態間遷移確率に対して境界条件を直接導入することで、反射境界および吸収境界を統一的に取り扱うことができる。

5.1.3 再帰確率 (Returning probability, RP) 理論との連携

RP 理論は、拡散律速反応を厳密に記述する理論であり、分子結合過程を反応状態 (reactive state) の熱力学的および動力学的性質によって特徴づける。IEPDYN 法では、RP 理論で必要となる動力学量を短時間 MD 計算から効率的に評価することができる。

RP 理論における結合の速度定数 k_{on} は、

$$k_{\text{on}} = k_{\text{ins}} K^* \left(1 + k_{\text{ins}} \int_0^\infty dt, P_{\text{RET}}(t) \right)^{-1}, \quad (5.8)$$

によって与えられる。ここで、 K^* は反応状態と解離状態との間の平衡定数、 k_{ins} は反応状態から結合状態への遷移速度定数、 $P_{\text{RET}}(t)$ は、初期時刻に反応状態に存在した系が時刻 t においても反応状態に存在する条件付き確率 (再帰確率) である。

IEPDYN 法では、反応状態の集合を $\mathcal{M}_{\text{R}}$ とし、初期状態を $\mathcal{M}_{\text{I}} = \mathcal{M}_{\text{R}}$ と設定する。さらに、適切な境界条件を導入することで、 k_{ins} および $P_{\text{RET}}(t)$ を状態存在確率 $P_j(t)$ から計算することができる。

k_{ins} の計算では、結合過程の終了を定義するために吸収境界条件を導入し、反応状態のうち解離側に位置する隣接状態に反射境界条件を課す。このとき、

$$\frac{1}{k_{\text{ins}}} = \int_0^\infty dt, \sum_{j \notin \mathcal{M}_{\text{absorb}}} P_j(t), \quad (5.9)$$

が成り立つ。右辺は、反応状態から吸収状態までの平均初到達時間 (Mean First Passage Time, MFPT) に対応しており、その逆数として k_{ins} が得られる。

一方、 $P_{\text{RET}}(t)$ の計算では、反応状態のうち結合状態側に位置する隣接状態を反射境界として設定する。このとき、 $P_{\text{RET}}(t)$ は

$$P_{\text{RET}}(t) = \sum_{j \in \mathcal{M}_{\text{R}}} P_j(t), \quad (5.10)$$

によって与えられる。

このように、IEPDYN 法では境界条件を適切に設定することで、RP 理論で必要となる k_{ins} および $P_{\text{RET}}(t)$ を短時間 MD 計算から効率的に評価できる。さらに、平衡定数 K^* と組み合わせることで、長時間スケールの分子結合過程に対する結合速度定数 k_{on} を算出することが可能となる。

5.1.4 $P_j(t)$ の時間積分に対する解析的評価

RP 理論における k_{ins} をはじめとして、速度論的なパラメータは、状態ポピュレーションの時間積分

$$\tau_j = \int_0^\infty dt P_j(t) \quad (5.11)$$

によって与えられる。吸収境界条件あるいは無限遠方を含む状態を導入することで、 $P_j(t)$ は $t \rightarrow \infty$ においてゼロへ収束する。通常は、積分方程式を逐次的に解いて $P_j(t)$ の時間発展を計算した後、その時間積分を評価する必要がある。しかし、IEPDYN 法では Laplace 変換を利用することで、 $P_j(t)$ を陽に計算することなく τ_j を直接評価することができる。

関数 $f(t)$ の Laplace 変換を

$$\tilde{f}(s) = \int_0^\infty dt e^{-st} f(t) \quad (5.12)$$

と定義すると、状態存在確率の積分方程式は

$$\tilde{P}_j(s) = \tilde{P}_j^0(s) + \sum_{k \in \mathcal{N}_j} \tilde{M}_{jk}(s) \tilde{Q}_{jk}(s) \quad (5.13)$$

と表される。さらに、 $s \rightarrow 0$ の極限を取ること、

$$\tau_j = \tilde{P}_j^0(0) + \sum_{k \in \mathcal{N}_j} \tilde{M}_{jk}(0) \tilde{Q}_{jk}(0) \quad (5.14)$$

を得る。したがって、 τ_j を求めるためには、 $\tilde{Q}_{jk}(0)$ を評価すればよい。そのために、

$$\mathcal{K}_{ij,j'k}(t) = \delta_{jj'} K_{ijk}(t) \quad (5.15)$$

を導入すると、 $Q_{ij}(t)$ に対する積分方程式は Laplace 空間において

$$\tilde{Q}_{ij}(s) = \tilde{R}_{ij}(s) + \sum_{j',k} \tilde{\mathcal{K}}_{ij,j'k}(s) \tilde{Q}_{j'k}(s) \quad (5.16)$$

と書き換えられる。ここで、状態対 (ij) を複合添字 λ で表すと、上式は

$$\tilde{\mathbf{Q}}(s) = \tilde{\mathbf{R}}(s) + \tilde{\mathbf{K}}(s) \tilde{\mathbf{Q}}(s) \quad (5.17)$$

と行列表記できる。したがって、 $s \rightarrow 0$ の極限では、

$$\tilde{\mathbf{Q}}(0) = [\mathbf{I} - \tilde{\mathbf{K}}(0)]^{-1} \tilde{\mathbf{R}}(0) \quad (5.18)$$

が得られる。ここで、 $\mathbf{I}$ は単位行列であり、

$$(\mathbf{I} - \tilde{\mathbf{K}}(0))_{\lambda\mu} = \delta_{\lambda\mu} - \int_0^\infty dt \mathcal{K}_{\lambda\mu}(t) \quad (5.19)$$

である。

以上より、時間積分された遷移確率 $\tilde{\mathbf{Q}}(0)$ を行列演算によって直接求めることができ、さらに τ_j を解析的に評価できる。この手法により、 $P_j(t)$ の長時間時間発展を逐次計算することなく、MFPT などの速度論的なパラメータを効率的に算出することが可能となる。

5.1.5 平衡ポピュレーション $P_j(\infty)$ の解析的評価

IEPDYN 法では、全ポピュレーションが保存される系に対して、状態 j の平衡ポピュレーション $P_j^{\text{eq}} \equiv P_j(\infty)$ を、 $P_j(t)$ の時間発展を数値積分することなく評価することができる。

まず、 $Q_{ij}(t)$ を、平衡成分 $Q_{ij}^{\text{eq}} \equiv Q_{ij}(\infty)$ と時間依存成分 $\delta Q_{ij}(t)$ に分解する：

$$Q_{ij}(t) = Q_{ij}^{\text{eq}} + \delta Q_{ij}(t). \quad (5.20)$$

ここで、 Q_{ij}^{eq} は状態 j から i への平衡状態におけるポピュレーション流束を表す。この分解を IEPDYN の積分方程式に適用し、 $t \rightarrow \infty$ の極限を取ると、

$$P_j^{\text{eq}} = \lim_{t \rightarrow \infty} \left[\sum_{k \in \mathcal{N}_j} \int_0^t d\tau M_{jk}(t - \tau) \delta Q_{jk}(\tau) \right] + \sum_{k \in \mathcal{N}_j} I_{jk} Q_{jk}^{\text{eq}}, \quad (5.21)$$

を得る。ただし、

$$I_{jk} = \int_0^\infty dt M_{jk}(t) \quad (5.22)$$

である。 $M_{jk}(t)$ および $\delta Q_{jk}(t)$ はいずれも $t \rightarrow \infty$ でゼロへ収束するため、第 1 項は消失し、

$$P_j^{\text{eq}} = \sum_{k \in \mathcal{N}_j} I_{jk} Q_{jk}^{\text{eq}} \quad (5.23)$$

が得られる。したがって、平衡遷移流束 Q_{jk}^{eq} が求まれば、各状態の平衡ポピュレーションを直接計算することができる。さらに、積分方程式を行列表記すると、

$$\delta \mathbf{Q}(t) = \mathbf{R}(t) + \int_0^t d\tau \mathbf{K}(t - \tau) \delta \mathbf{Q}(\tau) - \mathbf{Q}^{\text{eq}} + \mathbf{J} \mathbf{Q}^{\text{eq}}, \quad (5.24)$$

となる。ここで、

$$J_{\lambda\mu} = \int_0^\infty dt K_{\lambda\mu}(t) \quad (5.25)$$

である。 $t \rightarrow \infty$ の極限では、時間依存項が消失するため、

$$\mathbf{J} \mathbf{Q}^{\text{eq}} = \mathbf{Q}^{\text{eq}} \quad (5.26)$$

を得る。すなわち、 $\mathbf{Q}^{\text{eq}}$ は行列 $\mathbf{J}$ の固有値 1 に対応する固有ベクトルとして与えられる。したがって、 $\mathbf{J}$ の固有値問題を解くことで $\mathbf{Q}^{\text{eq}}$ を求め、さらに

$$P_j^{\text{eq}} = \sum_{k \in \mathcal{N}_j} I_{jk} Q_{jk}^{\text{eq}} \quad (5.27)$$

を用いて平衡ポピュレーションを評価することができる。得られた平衡ポピュレーションは、IEPDYN 法における初期統計重み

$$\forall j \in \mathcal{M}_1, \quad w_j = \frac{P_j^{\text{eq}}}{\sum_{k \in \mathcal{M}_1} P_k^{\text{eq}}} \quad (5.28)$$

の評価にも利用される。また、結合・解離過程では、最外殻状態に反射境界条件を導入することで全ポピュレーション保存系を構成できるため、本手法を適用して平衡ポピュレーションを効率的に求めることができる。

5.2 パラメーター一覧

Namelist	パラメータ	備考
&input_param	fcv	時系列 CV データファイル
	flist_cv	CV データファイル一覧
&output_param	fhead	出力ファイル名のヘッダ
&option_param	input_type	入力ファイル形式
	output_histogram	リスタートファイルの出力
	use_perturbed_traj	バイアスポテンシャル存在下の MD に基づくかどうか
	f_unperturbed_id	各データでどの状態のダイナミクスが乱されていないか
	use_dissociate_state	解離状態を定義するかどうか
	use_reflection_state	反射状態を定義するかどうか
	use_product_state	生成状態を定義するかどうか
	calc_steady	定常状態分布を計算するかどうか
	calc_Pint	時間積分を解析的に計算するかどうか
	extrapolate	積分方程式に基づく時間発展の評価
	nstate	状態数
	ndim	CV の次元数
	nmol	対象分子数
	dt	時間グリッド幅
	t_sparse	TCF 計算用の粗い時間グリッド幅
&state	t_range	K, M, R, P0 関数の計算時間範囲
	t_extend	P, Q 関数の出力時間範囲
	dt_tcfout	P, Q 関数の出力時間間隔
	initial_state_ids	初期状態 ID
	reflection_state_ids	反射状態 ID
&state	product_state_ids	生成状態 ID
	state definition	各状態の範囲および初期ポピュレーション

5.3 &option_param

• input_type

- Default: "TIMESERIES"
- Example: input_type = "TIMESERIES"

入力ファイル形式を指定する。利用可能な形式は

- TIMESERIES: ANATRA 標準の時系列データ
- HISTOGRAM: output_histogram = .true. により生成されたヒストグラムファイル (*.krhist)

である。

• output_histogram

- Default: .false.
- Example: output_histogram = .true.

計算結果からヒストグラム形式のリスタートファイル (*.krhist) を出力するかどうかを指定する。

• use_perturbed_traj

- Default: .false.
- Example: use_perturbed_traj = .true.

入力の時系列データがバイアスポテンシャル下での MD 計算により得られたものかどうかを指定する。フラットボトムポテンシャルなど、特定の状態内についてのダイナミクスを乱さないバイアスポテンシャルを用いる必要がある。 .true. の場合には、 f_unperturbed_id を指定する必要がある。

• f_unperturbed_id

- Default: N/A
- Example: f_unperturbed_id = "funp.list"

use_perturbed_traj = .true. の場合に指定する必要がある。各時系列データについて、(1) どの状態内でのダイナミクスがバイアスポテンシャルによって乱されていないか、(2) $R_{ij}(t)$ の計算に用いるか、を列挙したリストファイルを指定する。 flist_cv や fcv で列挙したファイルの順番に合わせる形で、1 行ごとに

- 1 列目: 乱されていない状態 ID
- 2 列目: $R_{ij}(t)$ の計算に利用するか (1), 利用しないか (0)

を記す。例えば、 f_unperturbed_id で指定したリストファイル funp.list が

```
1 1
1 1
2 0
2 0
```

となっている場合、最初の2つの時系列データは状態1のダイナミクスが乱されおらず、 $R_{ij}(t)$ の計算に用いる、ということの意味し、次の2つの時系列データは状態2のダイナミクスが乱されておらず、 $R_{ij}(t)$ の計算には用いない、ということの意味する。

- **use_dissociate_state**

- Default: `.false.`
- Example: `use_dissociate_state = .true.`

解離状態を定義するかどうかを指定する。

- **use_reflection_state**

- Default: `.false.`
- Example: `use_reflection_state = .true.`

反射状態を定義するかどうかを指定する。

- **use_product_state**

- Default: `.false.`
- Example: `use_product_state = .true.`

吸収状態（生成状態）を定義するかどうかを指定する。

- **calc_steady**

- Default: `.false.`
- Example: `calc_steady = .true.`

定常状態（平衡状態）における各状態の存在確率の解析的評価を実行するかどうかを指定する。

- **calc_Pint**

- Default: `.false.`
- Example: `calc_Pint = .true.`

各状態存在確率の時間積分の解析的評価を実行するかどうかを指定する。

- **extrapolate**

- Default: `.false.`
- Example: `extrapolate = .true.`

積分方程式を用いて、各状態存在確率 $P_j(t)$ の時間発展を計算するかどうかを指定する.

- **nstate**

- Default: `None`
- Example: `nstate = 4`

状態数を指定する.

- **ndim**

- Default: `1`
- Example: `ndim = 2`

Collective Variable (CV) の次元数を指定する.

- **nmol**

- Default: `1`
- Example: `nmol = 1`

対象とする分子数を指定する.

- **dt**

- Default: `None`
- Example: `dt = 0.1d0`

時間グリッド幅を指定する.

- **t_sparse**

- Default: `None`
- Example: `t_sparse = 0.1d0`

$K_{ijk}(t)$ などの時間相関関数 (TCF) の計算に用いる粗い時間グリッド幅を指定する.

- **t_range**

- Default: None
- Example: `t_range = 10.0`

$K_{ijk}(t)$, $M_{jk}(t)$, $R_{ij}(t)$, $P_j^0(t)$ を計算する時間範囲を指定する.

• `t_extend`

- Default: None
- Example: `t_extend = 100.0`

$P_j(t)$, $Q_{ij}(t)$ を出力する時間範囲を指定する.

• `dt_tcfout`

- Default: None
- Example: `dt_tcfout = 1.0`

$P_j(t)$, $Q_{ij}(t)$ を出力する時間間隔を指定する.

• `initial_state_ids`

- Default: None
- Example: `initial_state_ids = 2 3`

初期状態として扱う状態 ID を指定する.

• `reflection_state_ids`

- Default: None
- Example: `reflection_state_ids = 4`

反射状態として扱う状態 ID を指定する.

• `product_state_ids`

- Default: None
- Example: `product_state_ids = 1`

吸収状態（生成状態）として扱う状態 ID を指定する.

5.4 &state

&state セクションでは、各状態の CV 範囲と初期ポピュレーションを 1 行ずつ下記のように指定する。

```
lower_bound(CV1) upper_bound(CV1) lower_bound(CV2) upper_bound(CV2) ... lower_
↪bound(CVndim) upper_bound(CVndim) weight
```

- 1 列目：状態の CV1 の下限値
- 2 列目：状態の CV1 の上限値
- 3 列目：状態の CV2 の下限値
- 4 列目：状態の CV2 の上限値

...

- 最終列：状態の初期ポピュレーション

このように、下限と上限のセットは `ndim` で指定した分だけ必要となる。例えば、`ndim=1` の場合に、

```
&state
-500.0 -16.0 0.5
-16.0 -15.0 0.5
-15.0 -2.0 0.0
-2.0 500.0 0.0
/
```

と指定すると、4 つの状態が定義され、最初の 2 状態に初期ポピュレーション 0.5 が割り当てられる。

5.5 補助ツール

`iepdyn` には多くの設定が存在する。そこで、下記にインプット作成を補助する Python スクリプトを用意している。

```
$ANATRA_PATH/utility/iepdyn_util
```

以下の 2 つのスクリプトはいずれも対話型であり、引数なしで利用できる。

- `iepdyn_setup.py`
iepdyn 用のインプットを作成するための補助ツール。
- `cvlist_generator.py`
時系列データのリストを作成するための補助ツール。バイアスポテンシャルによって摂動を含んでいる時系列データを用いた場合 (`use_perturbed_traj = .true.` の場合)、時系列データの分類をディレクトリ階層に基づいて行うことができる。

5.6 出力ファイル

- *.krhist (output_histogram = .true. の場合)

リスタート用の $K_{ijk}(t)$, $R_{ij}(t)$ に関する情報が格納されている。リスタートでは、複数の krhist ファイルを読み込ませることが可能である。このファイルに格納される情報は境界条件を課す前のものであり、境界条件の設定を変えた場合でも際計算することなく利用可能である。

- *.P0

- 1 列目：時刻
- 2 列目以降： $P_j^0(t)$

何列目がどの状態の $P_j(t)$ に対応しているかはファイル内にヘッダーとして記されている。

- *.Rij

- 1 列目：時刻
- 2 列目以降： $R_{ij}(t)$ 何列目がどの $R_{ij}(t)$ に対応しているかはファイル内にヘッダーとして記されている。

- *.Mij

- 1 列目：時刻
- 2 列目以降： $M_{ij}(t)$

何列目がどの $M_{ij}(t)$ に対応しているかはファイル内にヘッダーとして記されている。

- *.Kijk

- 1 列目：時刻
- 2 列目以降： $K_{ijk}(t)$

何列目がどの $K_{ijk}(t)$ に対応しているかはファイル内にヘッダーとして記されている。

- *.pj (extrapolate = .true. の場合)

- 1 列目：時刻
- 2 列目以降： $P_j(t)$

何列目がどの $P_j(t)$ に対応しているかはファイル内にヘッダーとして記されている。

- *.tcf (extrapolate = .true. の場合)

- 1 列目：時刻
- 2 列目以降： $\sum_{k \in \mathcal{M}_I} P_k(t)$

- *.q (extrapolate = .true. の場合)

- 1 列目：時刻
- 2 列目以降： $Q_{ij}(t)$

何列目がどの $Q_{ij}(t)$ に対応しているかはファイル内にヘッダーとして記されている。

- *.int (calc_Pint = .true. の場合)

下記のフォーマットで

$$\tau_j = \int_0^\infty dt P_j(t), \quad (5.29)$$

が格納されている。

Pint	InitTotal	0.5549862E+02
Pint	1	0.5474586E+02
Pint	2	0.6403562E+00
Pint	3	0.9158297E-01
Pint	4	0.1578925E-01
Pint	5	0.5035148E-02
Pint	6	0.7000565E-02
Pint	7	0.3941343E-01
Pint	8	0.6599485E-01

2 列目は状態インデックスを表しており, 3 列目が τ_j の値である。InitTotal は, 初期状態 $\mathcal{M}_1$ に属する状態に対する τ_j の和である。

- *.steady (calc_steady = .true. の場合)

- 1 列目: 時刻
- 2 列目: 状態インデックス (j)
- 3 列目: P_j^{eq}
- 4 列目: $-k_B T \log \frac{P_j^{\text{eq}}}{\sum_{k \in \mathcal{M}_1} P_k^{\text{eq}}}$

5.7 解析例

分子間距離 r で規定される分子結合/解離を例として IEPDYN 法の適用手順について述べる。ここでは, 状態を下記のように設定する。

- 状態 1(結合状態): $0 \leq r < 2$
- 状態 2(結合状態): $2 \leq r < 3.5$
- 状態 3: $3.5 \leq r < 4$
- 状態 4 (解離状態を含む): $4 \leq r$

また、求める計算量を，residence time correlation function

$$P_B(t) = \frac{\sum_{k \in \mathcal{M}_I} P_k(t)}{\sum_{k \in \mathcal{M}_I} P_k(0)} \quad (5.30)$$

およびその積分値 τ_B と定める． τ_B は結合状態の寿命を表す．これらを求めるときには，結合状態 1, 2 が初期状態となる．

5.7.1 バイアスポテンシャルなしの MD トラジェクトリを利用する場合 (use_perturbed_traj = .false.)

5.7.1.1 リスタートファイル (krhist) の作成

まずは，時系列データごとにリスタートファイル (krhist) を作成する．時系列データ 0001.ts を読み込んで，krhist/0001.krhist を作成する場合のインプットを下記に示す．状態 1 と 2 が結合状態 (初期状態) であるが，krhist ファイル作成の際には，初期ポピュレーション値は参照されないので，&state でのこれらの状態の初期ポピュレーション値は 1.0 にしておいて良い．

```
#!/usr/bin/env bash

mkdir -p krhist

cat << EOF > run0001.inp
&input_param
  fcv = "0001.ts"
/
&output_param
  fhead = "krhist/0001"
/
&option_param
  input_type          = "TIMESERIES"
  output_histogram    = .true.
  use_perturbed_traj  = .false.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state   = .false.
  calc_steady         = .false.
  calc_Pint           = .false.
  extrapolate         = .false.
  nstate              = 4
  ndim                = 1
  nmol                = 1
```

(次のページに続く)

(前のページからの続き)

```

dt                = 2e-05
t_sparse          = 2e-05
t_range           = 1.0
initial_state_ids = 1 2
dissociate_state_ids = 4
/
&state
0.0      2.0      1.0
2.0      3.5      1.0
3.5      4.0      0.0
4.0      1.0d10   0.0
/
EOF

fanatra iepdyn run0001.inp

```

5.7.1.2 平衡ポピュレーション値の計算

$P_B(t)$ を IEPDYN 法により計算するためには、状態 1, 2 の初期ポピュレーション値が必要となる。そこで、解離状態 4 を反射状態と定めることで、状態 1-3 でポピュレーションが保存されるようにした上で、一つ前で計算したリスタートファイル (krhist) をもとに、平衡状態のポピュレーション (P_j^{eq}) を計算する。そのために、

- calc_Pint = .true.
- use_reflection_state = .true.
- reflection_state_ids = 4

と設定した上で、

- use_dissociate_state = .false.

に設定する。

```

#!/usr/bin/env bash

mkdir -p steady

# Create list of krhist files
#
ls krhist/*.krhist > krhist.list

# Create input file
#

```

(次のページに続く)

```

cat << EOF > steady.inp
&input_param
  flist_cv = "krhist.list"
/
&output_param
  fhead = "steady/out"
/
&option_param
  input_type          = "HISTOGRAM"
  output_histogram    = .false.
  use_perturbed_traj  = .false.
  use_dissociate_state = .false.
  use_reflection_state = .true.
  use_product_state   = .false.
  calc_steady         = .true.
  calc_Pint           = .false.
  extrapolate         = .false.
  nstate              = 4
  ndim                = 1
  nmol                = 1
  dt                  = 2e-05
  t_sparse            = 2e-05
  t_range             = 1.0
  initial_state_ids   = 1 2
  reflection_state_ids = 4
/
&state
0.0    2.0    1.0
2.0    3.5    1.0
3.5    4.0    0.0
4.0    1.0d10  0.0
/
EOF

fanatra iepdyn steady.inp

```

これを実行すると、steady/out.int が得られ、その中身は、例えば下記のようなものである。

1	0.9874940E+00	0.7502598E-02
2	0.1066799E-01	0.2706875E+01
3	0.1489986E-02	0.3880407E+01

2 列目が平衡ポピュレーションの値である。

5.7.1.3 $P_B(t)$ の計算

1 つ前のステップで得た平衡ポピュレーション値を&state 内の状態 1, 2 について入力した上で,

- extrapolate = .true.
- t_extend = 10.0

を指定する. t_extend は $P_j(t)$ を計算する時間範囲を規定する. また,

- use_dissociate_state = .true
- use_reflection_state = .false.
- dissociate_state_ids = 4

に設定しておく.

なお, 初期状態の平衡ポピュレーション値の和が 1 になるようにプログラム内で処理される.

```
#!/usr/bin/env bash

mkdir -p extrapolate

# Create list of krhist files
#
ls krhist/*.krhist > krhist.list

# Create input file
#
cat << EOF > extrapolate.inp
&input_param
  flist_cv = "krhist.list"
/
&output_param
  fhead = "extrapolate/out"
/
&option_param
  input_type          = "HISTOGRAM"
  output_histogram    = .false.
  use_perturbed_traj  = .false.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state   = .false.
  calc_steady         = .false.
  calc_Pint           = .false.
  extrapolate         = .true.
  nstate              = 4
```

(次のページに続く)

(前のページからの続き)

```

ndim          = 1
nmol          = 1
dt            = 2e-05
t_sparse      = 2e-05
t_range       = 1.0
t_extend      = 10.0
dt_tcfout     = 2e-05
initial_state_ids = 1 2
dissociate_state_ids = 4
/
&state
0.0    2.0    0.9874940E+00
2.0    3.5    0.1066799E-01
3.5    4.0    0.0
4.0    1.0d10  0.0
/
EOF

fanatra iepdyn extrapolate.inp

```

これを実行すると、出力ファイルの1つとして `extrapolate/out.tcf` が得られるが、このファイル内に $P_B(t)$ の情報が格納されている。

5.7.1.4 $P_B(t)$ の時間積分の解析的評価

今度は、 $P_B(t)$ の時間積分を、解析的方法により求める。この場合は、1つ前のステップでのインプットのうち、下記の変数を変更する。

- `extrapolate = .false.`
- `calc_Pint = .true.`

また、`t_extend` や `dt_tcfout` などは結果に影響を与えないが、削除しておく。

```

#!/usr/bin/env bash

mkdir -p Pint

# Create list of krhist files
#
ls krhist/*.krhist > krhist.list

# Create input file

```

(次のページに続く)

(前のページからの続き)

```

#
cat << EOF > Pint.inp
&input_param
  flist_cv = "krhist.list"
/
&output_param
  fhead = "Pint/out"
/
&option_param
  input_type          = "HISTOGRAM"
  output_histogram    = .false.
  use_perturbed_traj  = .false.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state   = .false.
  calc_steady         = .false.
  calc_Pint           = .true.
  extrapolate         = .false.
  nstate              = 4
  ndim                = 1
  nmol                = 1
  dt                  = 2e-05
  t_sparse            = 2e-05
  t_range             = 1.0
  initial_state_ids   = 1 2
  dissociate_state_ids = 4
/
&state
0.0      2.0      0.9874940E+00
2.0      3.5      0.1066799E-01
3.5      4.0      0.0
4.0      1.0d10    0.0
/
EOF

fanatra iepdyn Pint.inp

```

これを実行すると、Pint/out.int が得られる。このファイル内の 1 行目の 3 列目が $P_B(t)$ の時間積分値である。

5.7.2 バイアスポテンシャルありの MD トラジェクトリを利用する場合 (use_perturbed_id = .true.)

バイアスなしの場合と手順はほぼ同様だが、用意する f_unperturbed_id で指定するリストの作成などが必要となり、手続きがやや煩雑となる。

5.7.2.1 リスタートファイル (krhist) の作成

ここでは、時系列データごとに krhist ファイルを作成する。状態 j のダイナミクスが乱されていない時系列データが、

```
ts/0001/run0001.dis
ts/0001/run0002.dis
...
```

のようにディレクトリ名 (XXXX=0001, 0002, ...) で分類されているとする。このディレクトリ構造をもとに、まずは flist_cv と f_unperturbed_id に対応するリストを下記の Python スクリプトから作成する。

```
$ $ANATRA_PATH/utility/iepdyn_util/cvlist_generator.py
```

実行すると、下記のように、情報の入力を求められるので、指示に従って入力していく。

```
=== File Search Utility ===

(1) Enter the root directory to search: ts
(2) Enter the target extension (e.g., txt): dis
(3) Enter the maximum search directory depth: 2

Searching files...

=== Matched Files ===

/path/to/ts/0001/run0001.dis
/path/to/ts/0001/run0002.dis
/path/to/ts/0002/run0001.dis
/path/to/ts/0002/run0002.dis
/path/to/ts/0003/run0001.dis
/path/to/ts/0003/run0002.dis
/path/to/ts/0004/run0001.dis
/path/to/ts/0004/run0002.dis

-----
```

(次のページに続く)

(前のページからの続き)

```
(4) Enter directory level counted from the deepest directory for grouping (default: 1): 1
```

```
=== Grouped Files ===
```

```
--- State 1: 0001 ---
```

```
/path/to/ts/0001/run0001.dis
```

```
/path/to/ts/0001/run0002.dis
```

```
--- State 2: 0002 ---
```

```
/path/to/ts/0002/run0001.dis
```

```
/path/to/ts/0002/run0002.dis
```

```
--- State 3: 0003 ---
```

```
/path/to/ts/0003/run0001.dis
```

```
/path/to/ts/0003/run0002.dis
```

```
--- State 4: 0004 ---
```

```
/path/to/ts/0004/run0001.dis
```

```
/path/to/ts/0004/run0002.dis
```

```
-----
```

```
Did these files come from perturbed dynamics? (y/n): y
```

```
Available states:
```

```
1 : 0001
```

```
2 : 0002
```

```
3 : 0003
```

```
4 : 0004
```

```
Enter state numbers used for Rij calculation (space-separated, empty input allowed): 1 2
```

```
-----
```

```
Enter fraction of files to be listed from each state (default: 1.0 (corresponding to all
↳ the files)): 1.0
```

```
=== Output Preview ===
```

```
/path/to/ts/0001/run0001.dis
```

```
1 1
```

```
/path/to/ts/0001/run0002.dis
```

```
1 1
```

(次のページに続く)

(前のページからの続き)

```

/path/to/ts/0002/run0001.dis
2 1
/path/to/ts/0002/run0002.dis
2 1
/path/to/ts/0003/run0001.dis
3 0
/path/to/ts/0003/run0002.dis
3 0
/path/to/ts/0004/run0001.dis
4 0
/path/to/ts/0004/run0002.dis
4 0

```

```

-----

Enter filename to save the file list: flist
Enter filename to save state_id and Rij_flag: funp

```

この操作により、時系列データのリスト `flist` と各時系列データのどの領域が乱されていないかなどを記した `funp` が得られる。

次に、時系列データごとに `krhist` を計算するために、作成した `flist` と `funp` を下記の Python スクリプトにより 1 行ずつのファイルに分割する。

```
$ $ANATRA_PATH/utility/iepdyn_util/split_cv.py --flist flist --funp funp
```

上記を実行することで、下記のディレクトリとファイルが生成される。

```

0001/0001.flist
0001/0001.funp
0001/0002.flist
0001/0002.funp
0002/0001.flist
0002/0001.funp
0002/0002.flist
0002/0002.funp
0003/0001.flist
0003/0001.funp
0003/0002.flist
0003/0002.funp
0004/0001.flist
0004/0001.funp
0004/0002.flist
0004/0002.funp

```

生成された XXXX.flist, XXXX.funp が 1 つの時系列データに対する flist_cv, f_unperturbed_id である。したがって、これらを順次読み込んで krhist を作成すれば良い。

```
#!/usr/bin/env bash

for i in {0001..0004};do
  for f in `ls $i/*.flist`;do
    num=`basename $f | sed -e "s/.flist//g"`
    flist=$i/${num}.flist
    funp=$i/${num}.funp

cat << EOF > $i/run${num}.inp
&input_param
  flist_cv = "$i/${num}.flist"
/
&output_param
  fhead = "$i/run${num}"
/
&option_param
  use_perturbed_traj    = .true.
  f_unperturbed_id     = "$i/${num}.funp"

  output_histogram     = .true.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state    = .false.
  calc_steady          = .false.
  calc_Pint            = .false.
  extrapolate          = .false.
  nstate               = 4
  ndim                 = 1
  nmol                 = 1
  dt                   = 2e-05
  t_sparse             = 2e-05
  t_range              = 1.0
  initial_state_ids    = 1 2
  dissociate_state_ids = 4
/
&state
0.0      2.0      1.0
2.0      3.5      1.0
3.5      4.0      0.0
4.0      1.0d10   0.0
/
```

(次のページに続く)

(前のページからの続き)

```
EOF
fanatra iepdyn $i/run${num}.inp >& $i/run${num}.out
done
done
```

これを実行することで、`krhist` ファイルが生成される。

```
0001/run0001.krhist
0001/run0002.krhist
0002/run0001.krhist
0002/run0002.krhist
0003/run0001.krhist
0003/run0002.krhist
0004/run0001.krhist
0004/run0002.krhist
```

5.7.2.2 平衡ポピュレーション値の計算

バイアスポテンシャルがない場合と同様に、状態 1, 2 の初期ポピュレーション値を反射境界条件を導入することで計算する。krhist からなるリストなどは再度 cvlist_generator.py を利用することで求めておく。下記は、実行時に入力が必要とされている部分のみを抜粋した。

```
(1) Enter the root directory to search: .
(2) Enter the target extension (e.g., txt): krhist
(3) Enter the maximum search directory depth: 2
(4) Enter directory level counted from the deepest directory for grouping (default: 1): 1
Did these files come from perturbed dynamics? (y/n): y
Enter state numbers used for Rij calculation (space-separated, empty input allowed): 1 2
Enter fraction of files to be listed from each state (default: 1.0 (corresponding to all
↳the files)): 1.0
Enter filename to save the file list: fkrhist.list
Enter filename to save state_id and Rij_flag: fkrhist.unp
```

```
#!/usr/bin/env bash

mkdir -p steady

# Create input file
#
cat << EOF > steady.inp
&input_param
flist_cv = "fkrhist.list"
```

(次のページに続く)

(前のページからの続き)

```

/
&output_param
  fhead = "steady/out"
/
&option_param
  use_perturbed_traj   = .false.
  f_unperturbed_id    = "fkrhist.unp"

  input_type           = "HISTOGRAM"
  output_histogram     = .false.
  use_dissociate_state = .false.
  use_reflection_state = .true.
  use_product_state    = .false.
  calc_steady          = .true.
  calc_Pint            = .false.
  extrapolate          = .false.
  nstate               = 4
  ndim                 = 1
  nmol                 = 1
  dt                   = 2e-05
  t_sparse              = 2e-05
  t_range              = 1.0
  initial_state_ids    = 1 2
  reflection_state_ids = 4
/
&state
0.0    2.0    1.0
2.0    3.5    1.0
3.5    4.0    0.0
4.0    1.0d10  0.0
/
EOF

fanatra iepdyn steady.inp

```

実行することで、平衡ポピュレーション値が格納されている `steady/out.steady` が得られる。

5.7.2.3 $P_B(t)$ の計算

$P_B(t)$ の時間発展を `extrapolate = .true.` と設定することで計算する.

```
#!/usr/bin/env bash

mkdir -p extrapolate

# Create input file
#
cat << EOF > extrapolate.inp
&input_param
  flist_cv = "fkrhist.list"
/
&output_param
  fhead = "extrapolate/out"
/
&option_param
  use_perturbed_traj    = .true.
  f_unperturbed_id     = "fkrhist.unp"

  input_type           = "HISTOGRAM"
  output_histogram     = .false.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state    = .false.
  calc_steady          = .false.
  calc_Pint            = .false.
  extrapolate          = .true.
  nstate               = 4
  ndim                 = 1
  nmol                 = 1
  dt                   = 2e-05
  t_sparse             = 2e-05
  t_range              = 1.0
  t_extend             = 10.0
  dt_tcfout           = 2e-05
  initial_state_ids    = 1 2
  dissociate_state_ids = 4
/
&state
0.0    2.0    0.9874940E+00
2.0    3.5    0.1066799E-01
3.5    4.0    0.0
```

(次のページに続く)

(前のページからの続き)

```

4.0      1.0d10    0.0
/
EOF

fanatra iepdyn extrapolate.inp

```

5.7.2.4 $P_B(t)$ の時間積分の解析的評価

下記を実行すれば良い。

```

#!/usr/bin/env bash

mkdir -p Pint

# Create input file
#
cat << EOF > Pint.inp
&input_param
  flist_cv = "fkrhist.list"
/
&output_param
  fhead = "Pint/out"
/
&option_param
  use_perturbed_traj   = .true.
  f_unperturbed_id     = "fkrhist.unp"

  input_type           = "HISTOGRAM"
  output_histogram     = .false.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state    = .false.
  calc_steady          = .false.
  calc_Pint            = .false.
  extrapolate          = .true.
  nstate               = 4
  ndim                 = 1
  nmol                 = 1
  dt                   = 2e-05
  t_sparse              = 2e-05
  t_range              = 1.0
  t_extend              = 10.0

```

(次のページに続く)

(前のページからの続き)

```
dt_tcfout          = 2e-05
initial_state_ids   = 1 2
dissociate_state_ids = 4
/
&state
0.0    2.0    0.9874940E+00
2.0    3.5    0.1066799E-01
3.5    4.0    0.0
4.0    1.0d10  0.0
/
EOF

fanatra iepdyn Pint.inp
```

実行後, Pint/out.int が得られる.